

Evaluating Diverse Feature Extraction Techniques of Multifaceted IoT Malware Analysis: A Survey

Zhuoyun Qian Hongyi Miao Cheng Zhang Yili Jiang Qin Hu Jiaqi Huang Fangtian Zhong*

Abstract—As IoT devices continue to proliferate, their reliability is increasingly constrained by security concerns. In response, researchers have developed diverse malware analysis techniques to detect and classify IoT malware. These techniques typically rely on extracting features at different levels from IoT applications, giving rise to a wide range of feature extraction methods. However, current approaches still face significant challenges when applied in practice. This survey provides a comprehensive review of feature extraction techniques for IoT malware analysis from multiple perspectives. We first examine static and dynamic feature extraction methods, followed by hybrid approaches. We then explore feature representation strategies based on graph learning. Finally, we compare the strengths and limitations of existing techniques, highlight open challenges, and outline promising directions for future research.

I. INTRODUCTION

The Internet of Things (IoT) refers to physical devices that can interact and exchange data through the Internet. Over the past two decades, IoT technologies have been increasingly applied across diverse domains, evolving from smart homes to smart cities, industrial IoT, 5G-enabled edge computing, and more recently, healthcare systems and wearable devices. Today, IoT devices are deeply integrated into both industrial production and personal life, driving rapid market growth. As of August 2025, the global IoT market size has reached USD 53.13 billion and is projected to grow to USD 114.54 billion by 2033. This rapid expansion highlights that IoT technologies have become embedded in critical infrastructures, making their security issues increasingly significant [1], [2].

However, due to the limited computing resources of IoT devices, the lack of standardization, and the immaturity of security mechanisms, numerous vulnerabilities have emerged. Attackers can exploit these weaknesses to launch botnet, DDoS, and malware attacks against personal and industrial IoT devices, leading to severe consequences such as information leakage and financial losses.

To mitigate these security threats, researchers have proposed a variety of approaches, including static, dynamic, hybrid, and learning-based analyses. These methods typically detect and defend against attacks or malicious programs in IoT environments by analyzing IoT applications, network traffic among IoT devices, or other behavioral patterns. Broadly, such approaches can be divided into two stages: feature extraction and processing, followed by malware or attack analysis. Among them, the efficiency of feature extraction and the ability to capture more representative features of IoT

programs and network behaviors remain important research directions.

To the best of our knowledge, we provide the first survey that comprehensively investigates the features employed across different analysis methods as well as their corresponding feature extraction techniques. We provide an in-depth discussion of various features, including both single-feature approaches and multi-feature combinations, in the context of IoT malware detection and classification. Moreover, we examine the application of machine learning and deep learning techniques based on graph features for IoT malware analysis. Finally, we highlight unexplored directions and outline promising avenues for future research in this field.

The survey is organized as follows. After a brief introduction to the IoT security domain, Section II reviews recent related surveys, compares them with our work, and presents our contributions along with commonly used evaluation metrics. Section III discusses the use of diverse features for malware analysis from four different perspectives. Section IV further explores potential future research directions in this field. Finally, Section V concludes the paper.

II. RELATED WORK

A. Differences with Previous Related Surveys

Previous surveys have often concentrated on specific aspects of malware analysis, with relatively narrow scopes. For example, [3] investigates the security threats in UAV-assisted IoT networks and summarizes the use of learning-based methods to address these threats. Similarly, [4], [5] review the applications of ML/DL-based approaches and federated learning in attack detection and malware analysis. In the same vein, [6] focuses primarily on dynamic analysis techniques, systematically discussing both the techniques themselves and their integration with machine/deep learning for malware detection, while highlighting IoT malware analysis as a future research direction. Distinct from these works, [7] provides a systematic exploration of malware detection from the perspective of data mining, covering a variety of feature extraction and selection methods as well as different learning-based classification and clustering approaches. In contrast, [8] presents a more comprehensive overview of malware detection, encompassing static and dynamic analysis methods, comparisons of static and dynamic features and their selection techniques, and an in-depth discussion of the strengths and weaknesses of ML (e.g., SVM, RF, KNN) and DL (e.g., CNN, RNN, Transformer). Moreover, [9] focuses specifically on IoT

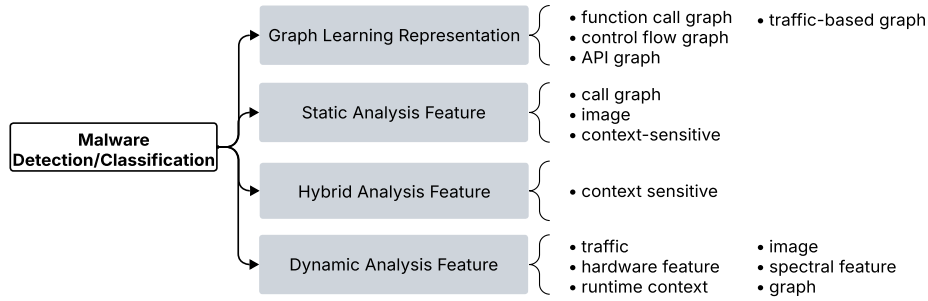


Fig. 1. Overview of the survey

malware detection, reviewing different analysis approaches and features, and providing a comprehensive survey of cross-architectural IoT malware threat hunting. Importantly, it also proposes a taxonomy that categorizes IoT malware features into four classes: metric-based, graph/tree-based, sequence-based, and interdependence-based, to address the challenges of future cross-architectural IoT malware detection.

In contrast to these surveys, our work fills the gap left by the lack of a systematic investigation into feature extraction techniques in IoT malware analysis. We provide a broader and more comprehensive perspective by focusing specifically on **Diverse Features** and **Diverse Feature Extraction Techniques**, aiming to inspire future research directions in IoT malware analysis. The differences between our survey and previous related surveys are presented in Table I.

B. Contributions

As shown in Figure 1, our survey categorizes the features used for malware analysis into four groups: *static analysis features*, *dynamic analysis features*, *hybrid analysis features*, and *graph learning representation features*. The key contributions of this survey are summarized as follows:

- 1) We present a comprehensive and modern feature-oriented overview of IoT malware analysis, covering diverse features, feature extraction techniques, and malware analysis methods. Our survey systematically examines static, dynamic, hybrid, and graph learning-based features for malware detection and classification.
- 2) We investigate various feature extraction, selection, and processing techniques, and compare the use of different feature categories. Special emphasis is placed on their differences, strengths, and limitations in different application scenarios.
- 3) We provide an in-depth discussion on both well-established research directions and emerging challenges in IoT malware analysis. The survey concludes by outlining future opportunities for leveraging different features in malware detection and classification.
- 4) To ensure comprehensiveness, we developed an automated Python script to retrieve relevant articles from ACM, IEEE, Springer, and Google Scholar. For each surveyed paper, we verified dataset and code availability, categorizing them as private, partially public, or public.

All collected data and our scripts are publicly released at xxxx.

C. Evaluation Metrics

Commonly adopted evaluation metrics in IoT malware detection and classification are listed as follows:

- **Accuracy:** Proportion of correctly classified samples over all samples.
- **Precision:** Proportion of true malware among all samples predicted as malware.
- **Recall / True Positive Rate:** Proportion of detected malware among all actual malware samples.
- **F1-score:** Harmonic mean of Precision and Recall, balancing false positives and false negatives.
- **Specificity / True Negative Rate:** Proportion of benign samples correctly identified as benign.
- **False Positive Rate (FPR):** Proportion of benign samples misclassified as malware.
- **False Negative Rate (FNR):** Proportion of malware samples misclassified as benign.
- **Resource Overhead:** Computational, memory, or energy cost during malware detection or classification.
- **Zero-day Detection Capability:** Ability to detect previously unseen malware families or variants.
- **Area Under ROC Curve (AUC-ROC):** Area under the ROC curve, measuring overall detection performance across thresholds.

III. MALWARE ANALYSIS ACROSS FEATURES

There are four primary features used to detect malware: static analysis, dynamic analysis, hybrid analysis, and graph representation learning. These methods collectively enhance the detection of malware by addressing different aspects and potential weak points in software security Figure 1.

A. Static Analysis Features

The workflow of static analysis-based malware detection and classification is illustrated in Figure 2. In general, existing approaches first process the input binary or bytecode using either customized algorithms or established tools (e.g., Radare2, IDA Pro). The processed results are then subjected to feature engineering to derive three major categories of features: *images*, and *context-sensitive features*, or are

TABLE I
COMPARISON OF EXISTING RELATED SURVEYS

Survey	Diverse Features	Diverse Feature Extraction Techniques	IoT Malware Analysis
[3]	X	X	✓
[4]	X	X	X
[5]	✓	X	✓
[8]	✓	✓	X
[7]	✓	✓	X
[9]	✓	X	✓
[6]	X	X	X

directly utilized as static features for the next stage. Alternatively, some methods take assembly code (e.g., opcode sequences) as input and extract features during the feature engineering stage through hashing or statistical analysis. Based on these features, malware detection and classification are ultimately performed using four categories of approaches: **machine learning-based, deep learning-based, clustering, and similarity analysis.**

1) *malware detection*: In the process of static malware detection, existing approaches typically rely on static analysis tools such as Androguard, FlowDroid, ApkTool, or dexdump to extract structural information directly from the input APK or binary file. These tools generate representations such as opcode sequences, or assembly code, while in some cases the raw byte-level structure itself is used as the input. Based on these representations, different methods further extract discriminative features; some studies even map the original program into images or convert assembly-derived features into image representations. Once obtained, these features or images are embedded into vector spaces using techniques such as Word2Vec, TF-IDF, sliding windows, or n -grams. Finally, the embedded vectors are fed into classifiers such as LSTM networks, CNNs, or ensemble models, which output the final detection result by classifying the input sample as either benign or malicious software.

Image: Converting a program directly into an RGB image and then applying deep learning methods to extract deep semantic features or spatial features from the image for malware identification is a common detection strategy. Some approaches further adopt feature fusion techniques by combining visual feature vectors extracted from images with semantic feature vectors directly obtained from the program, aiming to improve the accuracy of malware detection. ViT4Mal [10] follows the method proposed by Nataraj et al. [11] to convert the bytecode of an executable into an RGB image. It then applies a custom normalization formula to pixel values and resizes the image to a fixed dimension (128, 128, 3) via bilinear interpolation. The resized image is split into (8, 8, 3) patches, each treated as a token and fed into a lightweight Vision Transformer (ViT). The ViT encoder, composed of four layers, extracts features from each patch and outputs their corresponding feature vectors. These vectors are flattened into a single vector, which is fed into a multilayer perceptron (MLP) followed by a Softmax classifier to output the probability of malware or benign software.

In contrast, FED-MAL [12] first employs lightweight static analysis to extract three types of key information—permissions, components, and application programming interface (API) calls. A word embedding method is then used to encode this textual information into dense vectors, which are processed by a bidirectional independently recurrent neural network (Bi-IndRNN) to capture contextual dependencies and generate more representative semantic feature vectors. To obtain the visual feature vector, FED-MAL transforms the semantic feature vector into a grayscale image of size $N \times d$, where N denotes the number of features and d is the normalized length of each feature vector. This image is fed into a multilayer convolutional neural network (CNN), and after the max-pooling layer, the spatially-informed visual feature vector is obtained. Finally, the semantic and visual feature vectors are fused into a single capsule vector, which is passed through a Sigmoid function to produce the malware/benign prediction probability.

Context-sensitive features: Studies employing this type of feature can be broadly categorized into two groups. The first group leverages a single category of static features, which are directly extracted from the input executable file using analysis tools. Representative examples include opcode sequences, temporal-spatial attributes and shell command sequences. The second group adopts a multi-category feature fusion approach, where various types of features are first derived from static analysis outputs using techniques such as N-gram modeling, term frequency-inverse document frequency (TF-IDF), and byte entropy computation. These heterogeneous features are subsequently combined to provide a comprehensive representation of program behavior.

[13] directly accepts a bytecode sequence or an opcode sequence as input, whereas MTHAEL [14] employs a disassembler (the Object-Dump tool) to parse Executable and Linkable Format (ELF) binaries and extract opcode sequences from the instructions contained in the `.text` segment. After obtaining the bytecode or opcode, both approaches apply a sliding-window-based method to transform the sequence into vector embeddings, which are then either fed into a deep learning model for feature learning or further processed with dimensionality-reduction techniques for feature selection. In [13], for *bytecode*, each byte (0–255) is represented as a 256-dimensional one-hot vector. A non-overlapping sliding window of 1,024 bytes (1 KB) is applied, and only the first 100 windows (100 KB) are retained, yielding a sequence of

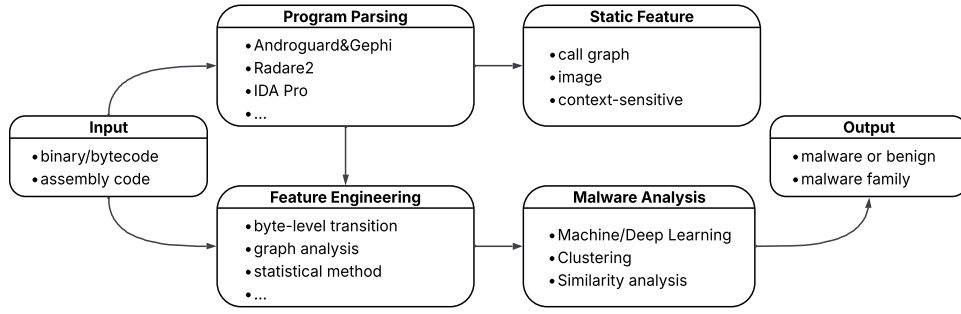


Fig. 2. Workflow of static analysis.

shape $(l = 100, m = 256)$. For *opcode*, one-hot encoding is first applied, followed by an embedding layer that maps the sparse vectors into 64-dimensional dense vectors (optimal size selected after testing values from 32 to 128), resulting in a sequence of shape $(l = 100, m = 64)$. These sequences are input into a stacked LSTM with unsupervised layer-wise pre-training: multiple pre-trained LSTM layers for abstract feature extraction, a vectorizing LSTM to compress the sequence into a fixed-length vector, and finally a dense layer with a softmax classifier to output malware/benign probabilities.

In contrast, MTHAEL [14] applies a sliding window to the opcode sequence to generate n -grams (e.g., 2-gram, 3-gram), and computes the frequency of each n -gram to obtain a high-dimensional sparse feature representation. Two feature selection techniques—Information Gain and an Opcode Dictionary—are then used to reduce dimensionality and retain the most representative features, producing compact feature vectors. These vectors are used to train two separate models, a Convolutional Neural Network (CNN) and a Recurrent Neural Network (RNN), whose outputs are subsequently integrated into a larger ensemble network for final malware detection.

The program’s temporal-spatial attributes constitute another important type of feature for representing program behavior. In the training phase, TSDroid [15] extracts two types of features from APK files: temporal features, which include the system APIs invoked by each app and their lifecycle information (i.e., the Android versions in which the APIs were added or deprecated), and spatial features, including apkSize (total APK file size), dexSize (size of classes.dex bytecode), and apiSize (number of system APIs used). These features are combined into a unified feature vector. TSDroid then applies the best-performing clustering algorithm—selected from K-Means, FCM, GMM, AGNES, and BIRCH—to group similar samples into clusters. Within each cluster, three detection models are trained: Drebin (static features + linear SVM), HinDroid (heterogeneous information network + multi-kernel learning), and DroidEvolver (ensemble of online models with weighted voting). The results of these models are fused to perform final malware detection. For an unknown APK, the same process is applied to extract features and assign the sample to a cluster, after which its feature vector is classified using the corresponding model to determine whether it is malicious.

ShellCore [16] focuses on extracting shell commands from software samples for malware detection. Given a sample, ShellCore first extracts a shell command sequence and applies two modeling strategies: a term-level model, which transforms the sequence into a bag-of-words and n -gram feature space, and a character-level model, where each character is treated as an individual token. Both models convert the shell command sequences into multihot vectors. To reduce dimensionality, ShellCore applies Principal Component Analysis (PCA) to both types of feature vectors. The reduced feature vectors extracted from the training set are then used to train malware classifiers such as Logistic Regression (LR), Random Forest (RF), and Deep Neural Networks (DNN). Once trained, these classifiers can determine whether a program is malicious based on the corresponding feature vector.

Compared with single-modal features, multi-modal features require more sophisticated processing of the original program to capture heterogeneous types of semantic information and integrate them for more effective malware detection. In this context, we primarily focus on the fusion of multi-modal semantic features and the fusion of multi-modal statistical features.

Both Fed-IIoT [17] and [18] focus on extracting three types of static features—API calls, permissions, and intents—from Android IoT applications and embedding them into vector representations for subsequent deep learning-based malware detection. The key difference lies in their feature encoding and utilization strategies. Fed-IIoT [17] encodes these features into a binary vector, where each dimension corresponds to a specific static feature, with a value of 1 indicating the presence of the feature and 0 otherwise. These vectors, together with adversarial samples, are used to train local models. After local training, model updates are transmitted to a central server, where an Attention-based Adaptive Adversarial Generative Network (A3GAN) computes anomaly scores for each participant; participants whose scores exceed a predefined threshold are flagged as potential poisoning attacks. In contrast, [18] directly feeds the three feature types into a TSA-Caps model, which encodes them into a unified multimodal feature vector. This vector is then passed to a pre-trained deep neural network for final classification to determine whether the APK is malicious.

In [19], Cypider first uses dexdump to reverse input apks

into bytecode. Then it extracts a diverse set of features from the bytecode, including opcode and byte-level n-grams from the `classes.dex` file, class and method names from disassembled code, API usage patterns, permissions, and component declarations from the `AndroidManifest.xml` file. Additionally, resource file names and their hashed contents, as well as printable strings weighted by TF-IDF, are also utilized. These heterogeneous features are transformed into vector representations through feature hashing. To capture relationships between APKs, Cypider constructs a similarity network using Locality Sensitive Hashing (LSH), where nodes represent APKs and edges indicate feature-level similarity between applications. This network is then used to train a One-Class Support Vector Machine (OC-SVM) for malware detection.

In contrast to the aforementioned works, [20] and [21] focus on further deriving statistical representations from context-sensitive features for malware detection. Specifically, [20] extracts five types of static features: Byte Histogram Features, Byte Entropy Histogram Features, String Information Features, TF-IDF Features (Term Frequency-Inverse Document Frequency), and Heterogeneous Graphs (HGs). The first four are statistical features, which are normalized and concatenated into vector representations, while the fifth, HGs, represents semantic features that are embedded into vectors using the `metapath2vec` algorithm. These feature vectors are used to train four separate models: a Gradient Boosting Decision Tree (GBDT) for byte histogram and string features, a Backpropagation Neural Network (BPNN) for byte entropy histogram features, a Random Forest for TF-IDF features, and a Support Vector Machine (SVM) with a polynomial kernel for semantic features. The outputs of these models are then combined via another GBDT to perform the final malware detection.

In contrast, [21] proposes a counterfeit app detection method driven by similarity analysis against a whitelist of verified official applications. For a given input app, three types of features are extracted to compute its similarity to whitelist entries: (1) TLSH-based Whole-App Similarity, which converts the entire app into a fixed-length hash for comparison; (2) Minhash-based Resource Similarity, which transforms resource identifiers into a matrix and computes similarity using standardized Euclidean distance; and (3) Motif-based Code Similarity, which extracts a Function Call Graph (FCG), transforms it into a motif frequency distribution (MFD) vector, and measures similarity based on motif type and frequency differences. The combination of these three similarity measures enables effective detection of counterfeit applications.

2) *malware classification*: IoT malware classification primarily relies on four common types of feature representations: **graph**, **image**, **context-sensitive**, and a **combination of image and context**.

Graph: In [22], the authors investigate the performance of similarity analysis based on three different graph embedding techniques for multi-platform (X86, ARM, MIPS) malware classification. Given a dataset containing samples from multiple malware families, their approach first employs

the Capstone disassembly engine to extract architectural information and executable code sections from each binary. Using a linear sweep strategy combined with data line filtering, they construct the corresponding control-flow graphs (CFGs). On these CFGs, three types of embeddings are explored: node embeddings with Node2Vec and VERSE, edge embeddings obtained by applying the Hadamard product on pairs of node vectors, and graph embeddings generated by Graph2Vec. Experimental results show that graph-level embeddings capture global structural features more effectively than the other two methods. Once graph embeddings are obtained, the authors classify malware samples either by averaging the embedding vectors and applying cosine similarity, or by performing pairwise similarity computation using their proposed Greedy Algorithm, which groups together the most similar malware instances.

Image: Existing approaches primarily construct malware images through four distinct strategies: (1) direct byte-to-pixel mapping, (2) disassembly-based mapping, (3) multi-modal mapping that combines byte-level and disassembly-derived contextual information, and (4) statistics-driven mapping. Once the images are generated, they are fed into a pre-trained machine learning (ML) or deep learning (DL) model, and the final malware classification is determined using a voting mechanism.

In both [23] and [24], malware binaries are first transformed into images through direct byte-to-pixel mapping, where each byte value corresponds to a pixel intensity. In [23], the raw binary is directly converted into a grayscale image, which is then processed by a deep neural network (ResNet50). ResNet50 maps the input into a 2048-dimensional feature vector. At the same time, a reference dataset containing N images from M malware families is also passed through ResNet50 to obtain their corresponding feature vectors. These feature vectors are subsequently transformed into binary hash codes, and similarity between samples is measured using the Hamming distance. Finally, malware classification is achieved via majority voting or weighted voting among the k nearest neighbors retrieved from the reference set. In contrast, [24] first applies the `procdump` tool to generate memory dump files from binaries, which are then mapped into grayscale images. Unlike [23], these grayscale images are further converted into color images using a colormap, and resized to fixed dimensions (e.g., 224×224 or 300×300). The normalized color images are then fed into a federated learning-based 2D CNN framework composed of convolutional, pooling, dropout, and fully connected layers. The CNN extracts a deep feature vector from the input image, which is directly passed to a softmax layer to output the final malware classification result.

Unlike directly mapping binaries into grayscale images, [25] first employs IDA Pro to extract three key types of information: binary code, assembly instructions, and developer-related metadata (represented as visible strings, such as calling conventions and compiler optimization options). These three categories of features are then mapped into the R, G, and B channels, respectively, to construct an RGB image. The

generated RGB image is subsequently processed through a Self-Attention module, which captures global dependencies among pixels, followed by a CNN module that extracts high-dimensional representations through multiple convolutional and non-linear activation layers. The resulting feature maps are then passed into a Spatial Pyramid Pooling (SPP) module to unify them into a fixed-size vector representation. Finally, the output vector is fed into a Softmax layer to produce the probability distribution over malware classes, thereby achieving malware classification and detection.

Furthermore, Mal3S [26] integrates multiple feature-mapping strategies to unify heterogeneous static features into image representations. Specifically, it extracts five types of static features from each malware sample: (1) raw byte sequences; (2) opcodes obtained from the `.text` section; (3) API calls also extracted from the `.text` section; (4) imported DLLs and their corresponding API calls from the `.idata` section; and (5) visible strings from the data section. In the image construction stage, byte sequences and strings are directly mapped into grayscale images. Opcodes and API calls are first embedded into vector representations using **FastText**, followed by **Min-Max normalization**, and then converted into corresponding images. For DLLs, Mal3S computes the frequency of API calls within each DLL to form an API call frequency vector, which is subsequently transformed into a frequency image. As a result, each malware sample is represented by five distinct images. These images are then fed into a multi-branch **Multi-SPP-net** (Multi-Scale Spatial Pyramid Pooling Network). The byte image is processed independently to generate a dedicated feature vector, while the remaining four images are fused and mapped into another feature vector. Finally, these two feature vectors are passed through a **Softmax** layer to yield the final malware classification.

The aforementioned studies share a common strategy: they extract static information directly from either assembly code or raw binary code, which is then mapped into image representations. Specifically, some approaches treat malware as a byte stream and generate Markov images by modeling the transition patterns of bytes [27], [28], while others disassemble binaries into opcode sequences and construct Markov images based on opcode transitions [29]. More concretely, in [27], each malware binary is represented as a byte stream, and multiple basic Markov images are constructed by calculating transition frequencies between consecutive byte pairs; these images are then merged into a composite image. In [28], the HEXDUMP tool is employed to convert the DEX file extracted from an APK into a hexadecimal sequence, from which state transition probabilities are computed for each byte, aggregated into a 256×256 Markov matrix, and finally visualized as a grayscale image. In [29], IDA Pro is used to disassemble PE-format binaries into opcode sequences, followed by the n-gram technique to calculate the transition frequencies of consecutive opcodes, producing a 2-gram Markov transition probability matrix that is subsequently converted into a grayscale Markov image. Once the images are constructed, they are fed into a

pre-trained Lightweight Convolutional Classification Network (LCCN) or a Convolutional Neural Network (CNN) to extract deep feature vectors and ultimately predict the malware family or type.

Context-sensitive features: The extraction and utilization of context-sensitive features can be categorized into three types: the first extracts instruction and API information from opcodes or basic blocks; the second derives statistical features from opcodes or program bytes; and the third directly obtains file structure information from binary files.

SOMRN [30] first extracts the opcode and API call sequences from malware samples. These sequences are then segmented into multiple sliding windows, each containing a central word and a set of context words. The S-Net module in SOMRN employs a context-separated bi-directional long short-term memory (CS-Bi-LSTM) model to process each window and generate its corresponding semantic vector. Moreover, SOMRN incorporates a Vector Quality Estimator to evaluate the quality of the generated semantic vectors. If the quality score exceeds a predefined threshold, the vector is passed to a classifier that predicts the malware category. During training, if the vector quality score falls below the threshold—indicating unclear semantics—an agent module is activated to guide and optimize the CS-Bi-LSTM, thereby improving the quality of semantic representation. Building on a similar idea of leveraging instruction-level semantics, EVOLIoT [31] disassembles collected malware binaries by using IDA pro and extracts basic blocks from the resulting assembly code. A customized instruction normalization strategy is then applied to convert each basic block into a token sequence. These token sequences are subsequently fed into a pre-trained BERT model to obtain semantically rich vector representations, with each malware sample being transformed into a vector. These vectors are then passed to a BERT encoder trained under a contrastive learning framework, enabling clustering of the unlabeled training set. For the resulting malware clusters, the authors perform manual analysis using regular expressions to identify behavioral characteristics specific to each cluster.

Complementing these context-sensitive sequence-based approaches, [32] applies CBOW Word2Vec to encode both the raw bytes of a malware binary and its disassembled instructions into a $256 \times 256 \times 2$ matrix representation. This matrix is then fed into a pre-trained Multilayer Perceptron (MLP) model, which performs classification to predict the malware type. Moreover, in [33], the proposed tool takes as input a malware sample consisting of a BYTE file and an ASM file, from which it extracts four types of features: (1) byte-level features, including Byte Histogram, Byte Entropy Histogram, and Printable String Statistics; (2) format features, such as field size statistics; (3) TF-IDF vectors derived from strings and assembly sequences using 1- to 3-gram representations; and (4) function-level assembly embeddings generated via Word2Vec. These four types of feature vectors are then processed using three different multimodal fusion strategies to obtain a set of fused representations. The resulting vectors are passed

through an Extra Trees Classifier based on Gini importance to select the top-k most informative vectors. Finally, the selected vectors are fed into a pre-trained XGBoost classifier, which outputs the predicted malware type.

MFSRC [34] derives statistical features from opcodes. It first employs IDA Pro version 6.5 to extract opcode sequences from malware samples, and then utilizes a customized tool to generate opcode frequency vectors. During training, these vectors are transformed into low-dimensional fuzzy relevance vectors using a fuzzy transformation technique. For each malware family, MFSRC computes a centroid vector by measuring Euclidean distances among the fuzzy vectors. It then applies fuzzy clustering by computing cosine similarities to group similar malware families and determine a similarity threshold for each family. This process is iteratively refined until optimal centroid vectors and corresponding fuzzy similarity thresholds are obtained. Once training is complete, MFSRC generates a fuzzy relevance vector for a newly input malware sample and uses the trained model to predict its classification probabilities.

Unlike the above approaches that primarily focus on byte-level or instruction-level features, [35] simultaneously considers both ELF file-level features and opcode-level features. Specifically, a static analysis approach is applied to extract two types of numerical features from ELF files, where the files are disassembled using Radare2. The first type corresponds to ELF file attributes, including instruction set architecture (ISA), file size, library usage, packing indicators, function counts, networking behavior, and YARA rule matches. The second type captures opcode features, such as logic operations, memory access, stack operations, arithmetic, control and status operations, procedure calls, system I/O, system control, branching, execution timing, prefixed instructions, and miscellaneous operations. These two feature sets are then independently fed into separate CNN-based models: one trained to distinguish malware from benignware, and the other to classify the detected malware into specific families.

Image and context sensitive: The paper combines image-based features with context-sensitive features for malware classification. In [36], the authors adopt the method from [11] to convert the input malware binary into a grayscale image. Simultaneously, they apply Natural Language Processing (NLP) techniques to the same binary to extract the top 50,000 most frequent byte-level "words," which are then embedded into a vector space. Based on these two types of features, the authors train a fusion model composed of a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network. An image-based module processes the grayscale image, while a string-based module processes the embedded vectors, each generating deep feature representations. These features are then fused into a joint feature vector, which is fed into a neural network, and the final classification result is produced by the output layer.

3) *Lessons learned:* This survey investigates the use of different static features and their extraction techniques in malware analysis. In the following, we highlight the key lessons learned.

- Current static malware detection methods mainly rely on image features and context-sensitive features. For image-based approaches, there are two common strategies: the traditional byte-mapping method, which directly converts the entire program or code segments into grayscale or RGB images, and the static analysis combined with deep learning method, where semantic feature vectors are first extracted and subsequently transformed into images. The former is simple to implement but often fails to capture effective features in obfuscated programs, whereas the latter, though more complex, can effectively mitigate the introduction of noise into feature vectors before they are fed into the classifier. Detection methods based on context-sensitive features can be broadly categorized into three groups: single-category features, multimodal features, and statistical features. Single-category features (e.g., bytecode, opcodes, temporal-spatial attributes, and command sequences) are typically obtained using disassembly tools, sliding windows, or n-gram techniques. However, such features are often insufficient to comprehensively represent program behavior, are prone to noise during detection, and exhibit high computational complexity, limiting their applicability to large-scale or complex programs. A promising future direction is the fusion of multiple single-category features into multimodal feature representations, enabling more accurate characterization of program behavior. Statistical features, by contrast, are mainly derived through hashing or mathematical computations, capturing statistical properties such as byte distribution and entropy. The differences of the reviewed features in malware detection is shown in Table II.
- Current static malware classification techniques primarily focus on three categories of features: graph-based, image-based, and context-sensitive features. Research leveraging graph features is relatively limited, and only a few studies have explored similarity analysis for malware classification. Similarly, the integration of image and context-sensitive features for classification remains an underexplored direction. For image construction, existing works typically adopt three strategies: directly mapping byte streams into images, extracting semantic features via static analysis and subsequently mapping them into images, or computing byte transition probabilities and n-gram transition probabilities to generate Markov images. In contrast, the extraction techniques for context-sensitive features are commonly shared across both malware detection and classification studies, highlighting their broader applicability. The differences of the reviewed features in malware classification is shown in Table III.

B. Dynamic analysis Features

Dynamic analysis has become a common and effective approach for malware detection and classification. A general workflow of dynamic analysis is shown Figure 3. Existing studies typically monitor running IoT devices to collect diverse

TABLE II
STATIC ANALYSIS FEATURES FOR MALWARE DETECTION

Paper	Feature Extraction	Performance
Image		
[10]	Bytecode map	96.2%
[12]	static analysis+Bi-IndRNN	83.5%, 82.2%
Context sensitive feature		
[13]	-	94.1%, 96.0%, 88.0%
[14]	Object-Dump tool	98.6%
[15]	-	91.4%, 89.3%
[16]	-	98.1%, 0.16%, 0.13%
[17]	static analysis	95.7%
[18]	static analysis	99.6%, 99.6%
[19]	dexdump	87%
[20]	statistic computing	99.1%, 99.1%, 99.0%, 0.8%
[21]	similarity computing	98.4%

sources of dynamic information, including *system logs*, *sensor data*, *communication signals*, *network traffic*, and *runtime hardware information*. After feature engineering, these raw data are transformed into six major categories of features: **traffic**, **hardware features**, **runtime context**, **image**, **spectral features**, and **graph**. After that, these features can either be directly utilized by traditional techniques such as pattern matching and statistical analysis, or embedded into vector representations and fed into machine learning or deep learning models for malware detection and classification.

1) *malware detection*: Existing methods uses six types of feature for dynamically detecting IoT malware, including **traffic**, **hardware feature**, **runtime context**, **image**, **spectral feature**, **graph**.

Traffic: Several studies, including [37], [38], [39], [40], and [41], directly analyze either publicly available network traffic datasets or raw traffic captured in .pcap format using tools such as Wireshark, Tcpdump, and SnifferPro. For example, in [37], the authors employed an existing dataset containing features such as external IP addresses, TCP packets, and TCP urgent packets. After data cleaning, since all features were numerical, they were directly used as input to a hybrid deep learning model that integrates a Recurrent Neural Network (RNN) with Long Short-Term Memory (LSTM) layers and an Artificial Neural Network (ANN) with a softmax output layer. In contrast, [38] adopted a more refined feature selection strategy. After removing incomplete or missing entries, cross-correlation analysis was applied to identify 44 relevant features, which were further pruned to 29 numerical ones. These features were then used to train a three-layer RNN-LSTM model with backpropagation, producing binary predictions of malicious or benign traffic.

To improve representativeness and discriminability, [39] designed a comprehensive preprocessing pipeline that included data cleaning, domain knowledge-based feature engineering, down- and over-sampling, label encoding, and min-max normalization. This process yielded 91 numerical features covering multiple protocols (e.g., Frame, IP, ARP, ICMP, HTTP). The features were used to train local models at edge servers, with parameters aggregated via the FedAvg algorithm on the

cloud to build a global model, which was then redistributed for real-time detection at the edge. Similarly, [40] and [41] performed several preprocessing steps, including removal of local identifiers (e.g., IP addresses), normalization, class balancing, feature vectorization, and anonymization. In particular, [41] applied Pearson Correlation Coefficient (PCC) analysis to select the 25 most informative features, and addressed data imbalance using class weighting and down-sampling. The resulting vectors were then classified using models such as Support Vector Machines (SVM), Random Forests (RF), Deep Neural Networks (DNN), and even more complex architectures such as a seven-layer Multi-Layer Perceptron (MLP).

BOTA[42] first collects large-scale IP flow data using multiple ipfixprobe flow exporters deployed across different peering links. To handle asymmetric routing, some flows are monitored by multiple exporters and are reconstructed into extended bidirectional flows at the flow collector. These flows are then normalized and fed into the BOTA IDS. BOTA adopts a two-stage classification pipeline for malicious traffic detection. The first stage consists of weak indicators, where each low-level detector analyzes specific extended flow fields and outputs a Boolean value indicating whether certain benign or malicious behaviors are observed. The second stage employs a heterogeneous meta-classifier, which builds Boolean decision functions by running malware samples in a sandbox and combining weak indicator outputs using classification algorithms. If a flow matches any of the Boolean rules, the system flags the corresponding device as compromised and generates an interpretable alert. The alert includes the device ID, flow timestamp, triggered detectors, and the Boolean expression used by the meta-classifier. This approach integrates flow-based detection, Boolean logic composition, and sandbox-driven training to enable fine-grained malware identification.

Recent studies [43], [44], [45], [46] commonly follow a pipeline where raw network traffic is collected and transformed into structured feature representations through correlation analysis, statistical extraction, temporal-spatial modeling, or enriched flow-level attributes. These features are then utilized in various learning frameworks, such as fuzzy clustering, deep neural networks, matrix factorization, or autoencoder-based models, to distinguish between benign and malicious behaviors.

In [47], the authors first employ the CICFlowMeter tool to extract features—such as source IP, port, protocol, and byte count—from raw traffic packets (.pcap files), and store them in a .csv file. Columns with more than 30% missing values are removed, and the remaining features are grouped into three modalities: flow (traffic statistics), flag (control features like ACK and SYN), and packets (packet-level attributes). The dataset is then split into training and testing sets using an 80:20 ratio. Subsequently, Min-Max normalization is applied, followed by feature selection using recursive XGBoost and SULOV (Search for Uncorrelated List of Variables) to identify an optimal set of feature vectors. These vectors are used to train a deep learning model for classifying network traffic as malicious or benign.

TABLE III
STATIC ANALYSIS FEATURES FOR MALWARE CLASSIFICATION

Paper	Feature Extraction	Performance
Graph		
[22]	Capstone	65.4%
Image		
[23]	byte-to-pixel	96.5%, 96.0%, 95.7%
[24]	procdump+image map	96.9%, 96.9%, 96.3%
[25]	IDA Pro+image map	97.0%, 97.4%, 96.9%, 97.1%
[26]	image map	98.4%, 96.8%, 98.3%, 97.5%
[27]	byte transitions	97.3%, 13.7%
[28]	HEXDUMP+Markov transition	99.6%, 99.6%
[29]	IDA Pro+Markov transition	97.6%
Context sensitive feature		
[30]	static analysis	98.0%, 98.5%, 98.5%, 97.1%, 98.5%
[31]	IDA Pro	88.8%
[32]	-	98.7%, 98.4%, 98.4%
[33]	static analysis+statistic computing	99.6%, 99.0%
[34]	IDA Pro	95.5%, 88.1%, 85.7%
[35]	Radare2	95.7%, 98.9%, 98.9%, 97.7%
Context + Image		
[36]	byte map+NLP	99.7%, 99.5%

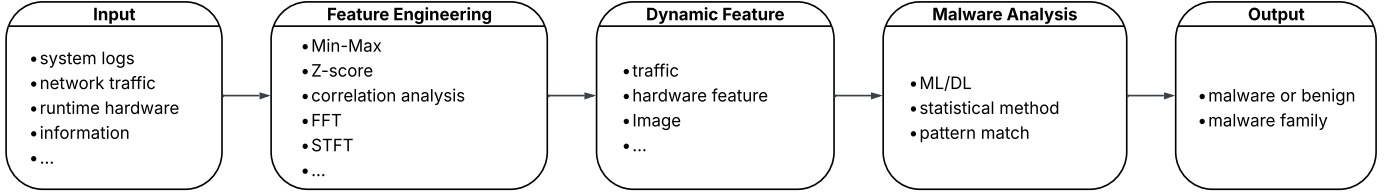


Fig. 3. Workflow of dynamic analysis.

In the clustering phase, IOT-KEEPER [43] first collects all network traffic from the training dataset and extracts 38 statistical features, including packet count, byte count, number of unique IPs and ports, connection duration, protocol type, and error flags (e.g., TCP resets). Redundant features are removed using correlation analysis combined with the Correlation-based Feature Selection (CFS) method. The remaining features are then clustered using Fuzzy C-Means (FCM), and the resulting clusters are used to generate fuzzy rules that map feature combinations to "benign" or "malicious" labels. In the fuzzy inference phase, for any newly observed traffic, the system performs rule matching and inference. If a rule is triggered, the traffic is classified as either malicious or benign based on the corresponding output.

In [44], it proposes an IoT-oriented DDoS attack detection method composed of four main stages. First, based on a real-world smart city dataset, the system constructs a traffic sequence for each node, denoted as $X_i = [x_i^{(1)}, \dots, x_i^{(T)}]$, where $x_i^{(t)}$ represents the number of packets sent within the t -th time window. In the second stage, two types of inputs are generated: the single-node sequence x_i and the combined sequence $[x_i, x_{N(i)}]$ with its neighboring nodes, enabling spatial correlation modeling. In the third stage, the extracted features are fed into five neural network models—Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), Transformer, and AutoEncoder—for training. Four configurations are evaluated to

compare the impact of using multiple models and incorporating neighbor information. In the fourth stage, correlation-based scoring methods such as Pearson, SHAP, and LIME are applied to identify key neighbors and reduce input dimensionality. During detection, the system extracts features from each node's traffic sequence, constructs inputs based on the training configuration, and feeds them into the deployed model to produce a binary classification output indicating whether an attack is occurring in the current time window.

This method [45] builds a malicious activity detection system based on federated Non-negative Matrix Factorization (NMF), comprising four stages. First, Darknet sensors deployed at different locations collect unsolicited traffic and construct observation matrices, with time steps as rows and IPs or ports as columns. An expansion function aligns these matrices across sensors and imputes missing values, enabling federated modeling without sharing raw data. Each aligned matrix is then factorized into temporal patterns (W) and spatial patterns (H) to capture latent features of coordinated scans. During training, clients compute local gradients and send intermediate results to a central server, which updates model parameters via a multiplicative update rule. In the final stage, suspicious IPs or ports are detected by analyzing the normalized H matrix: an alert is raised if any value exceeds a set threshold or activation exceeds 1. The approach achieves effective collaborative scan detection while preserving data privacy.

In [46], the authors propose a malicious traffic detection method that combines an autoencoder with a clustering mechanism. After collecting traffic data, the system represents it using the IP Flow Information Export (IPFIX) format and appends a rich set of features, including: raw IPFIX attributes (e.g., protocol type, packet count, flow duration), derived statistics (e.g., inter-arrival time), DNS-based features (e.g., the proportion of numeric characters in queried domain names), reputation intelligence from Webroot (e.g., IP risk categories), and time-dependent collaborative features (e.g., how frequently other devices of the same type accessed the same destination IP or port in the past hour). These features are first input into an autoencoder (AE) trained on benign traffic collected from multiple environments. If the mean squared error (MSE) of a new input exceeds a predefined threshold, the traffic is considered infrequent and is passed to a pre-trained K-Means clustering model. The model then classifies it as either a rare benign flow or a malicious one. Similarly, in [48], the proposed system first captures IoT traffic related to MQTT, DNS, and HTTP protocols using `tcpdump`. Protocol- and flow-level features—such as request-response pairs, the number of DNS queries, and TCP/IP metadata—are then extracted using Bro-IDS (now known as Zeek). A sliding window mechanism is applied to construct sequential features with temporal context. To reduce redundancy, the system computes the correlation coefficient (CC) between each pair of features and removes highly correlated ones. The resulting feature set is fed into a pre-trained AdaBoost ensemble framework that integrates three types of weak learners: Decision Trees (DT), Naive Bayes (NB), and Artificial Neural Networks (ANN). The ensemble outputs a final classification label for each traffic instance, indicating whether it is benign or malicious.

In [49], the authors propose a DNS traffic-based Advanced Persistent Threat (APT) malware detection framework tailored for IoT and Unmanned Aerial Vehicle (UAV) environments. The system first extracts `[device, domain, timestamp]` records from DNS request traffic, applies conditional filtering, and computes the time interval sequence between repeated requests to the same domain. Similar intervals are normalized to identical values, and the resulting sequence is mapped into a character string representation. Two complementary periodicity detection methods are then applied: a string-matching algorithm, suitable for patterns with explicit regularity, and a Fourier transform-based method, capable of detecting hidden periodicities embedded in noise. Each method outputs a confidence score, which are combined into a feature vector and fed into a pre-trained Random Forest (RF) model to produce the final malicious domain detection result.

LEDEM [50] is a learning-based mechanism for DDoS detection. It extracts features from network traffic, including protocol type, flow count, packet rate, and statistical information on source/destination IP addresses and ports. These features are transformed into vectorized tuples and fed into the DDoS detection module. The detection module employs a pre-trained Semi-supervised Stacked Deep Extreme Learning Machine (SDELM) as the classification model. SDELM com-

bines multiple autoencoder layers with an Extreme Learning Machine (ELM) classifier, and leverages both labeled and unlabeled samples during training. It uses a Gaussian kernel to compute sample similarity and constructs a graph Laplacian to preserve the neighborhood structure of the data. The final ELM classifier predicts whether the input traffic represents a DDoS attack based on the learned representations.

Unlike the above-mentioned work, [51] proposes a three-layer intrusion detection system that incorporates a privacy-aware, game-theoretic detection mechanism. The system first collects audit data (i.e., network behavior, including traffic logs) at the IoT layer. This audit data is then uploaded to the fog layer, where a signaling game named MPMDG is used to decide whether to preserve user privacy (by modifying the audit data before upload) or to directly forward the unaltered data to the cloud layer for intrusion detection. Finally, the cloud layer employs a deployed IDSaaS (Intrusion Detection System as a Service) to analyze the data received from the fog layer and determine whether malicious activity is present. This work focuses on investigating how the false positive rate at the fog layer and the detection rate at the cloud layer affect the system's ability to defend against the spread of malicious behavior.

Generally, existing IoT malware detection frameworks share a common workflow that can be summarized as: **data collection** from network gateways or edge devices, **feature extraction and processing** to transform raw traffic into structured representations, and finally **malware detection** through machine learning or deep learning models. Despite this shared pipeline, different systems emphasize distinct types of features and employ varied processing methods, which can be broadly categorized into symbolic representation, statistical/entropy-based features, temporal sequence features, distributional analysis, and autoencoder-driven representations.

In the category of **symbolic representation**, DIOT [52] collects network traffic between IoT devices and other network entities via a security gateway. Each packet is parsed to extract a 7-tuple representing seven features, including direction, port type, packet length, TCP flags, protocol type, and inter-arrival time. DIOT then uses AuDI to identify the device type associated with each traffic flow and assigns a symbolic label to it. For the traffic corresponding to each specific device type, DIOT applies a type-specific mapping function to embed the extracted 7-tuple into a discrete symbol s . As a result, the network behavior of an IoT device is transformed into a sequence of discrete symbols S , which is subsequently fed into a pre-trained type-specific GRU (Gated Recurrent Unit) model to determine whether the device's behavior is anomalous. In model training, DIOT also uses federated learning framework for training.

For **statistical and entropy-based features**, Adept [53] profiles each device using session-level statistics (e.g., mean and standard deviation of flow sizes), generates alert records for anomalous flows, and applies Frequent Itemset Mining (FIM) with traditional classifiers (k-NN, RF, SVM) for final-stage detection. ADAM [54] instead computes entropy vec-

tors from header-level traffic features to build a nominal profile under normal conditions, then applies unsupervised anomaly detection (KNN, K-means, Isolation Forest) during congestion-triggered sampling. Detected anomalies are mitigated by generating OpenFlow rules that progressively drop malicious traffic. He et al. [55] also fall into this category by extracting five-tuple and session-level statistics (e.g., duration, byte count, flags), applying z-score normalization, and training classifiers such as Decision Tree, Random Forest, KNN, and XGBoost. A distinct line of work employs **distributional and probing analysis**. LeakyPick [56] actively probes smart devices by playing audio and observing traffic bursts and statistical changes (e.g., packet size and inter-arrival times). Significant distributional differences are identified using burst detection and two-sample t-tests, revealing devices that may leak audio data. Finally, WADAC [57] combines **autoencoder-driven representations** with statistical features. It extracts packet-level and traffic-level attributes (e.g., Census, Ratio, Load, Gap), stores them in an SQLite database, and trains an autoencoder on benign traffic. During detection, feature vectors with reconstruction errors exceeding a threshold are flagged as anomalous, and Random Forest is then used to classify the specific attack type.

The use of **temporal sequence features** is exemplified by FIOT [58]. It first monitors IoT device traffic in real time through edge nodes and extracts 115 temporal statistical features from each packet using a Kitsune-based feature extractor (across five time windows ranging from 100 ms to 1 minute). These features are then reduced to 40 dimensions using Principal Component Analysis (PCA) and fed into a pre-trained Gated Recurrent Unit (GRU)-based neural network. The classifier consists of a unidirectional GRU layer for capturing temporal behavior, followed by a tanh activation function, a fully connected layer, and a softmax output that determines whether the traffic is malicious.

A number of recent studies emphasize the importance of **feature normalization and standardization** techniques in preparing IoT traffic data for malicious behavior detection. For instance, min-max normalization or scaling is widely adopted in [59], [60], [61], [62], [63], [64], [65], ensuring that heterogeneous traffic features are transformed into comparable numerical ranges. Specifically, [59] leverages the `MinMaxScaler` function. MalBoT-DRL [60] also employs min-max scaling. Similarly, [61], IoT-Sentry [62], [63], [64], and [65] incorporate Min-Max normalization in combination with other preprocessing techniques such as quantile transformation, digitization, SMOTE, or one-hot encoding.

More specifically, the method proposed in [61] deploys Host-based Intrusion Detection Systems (HIDS) and Network-based Intrusion Detection Systems (NIDS) to locally monitor audit logs and network packets, generating alerts containing packet metadata and audit information. These alerts are encrypted and uploaded via smart contracts to a permissioned consortium blockchain. Within the blockchain, the Central Coordination Unit (CCU) decrypts the alerts inside a Trusted Execution Environment (TEE) and verifies them using a Byzantine

Fault Tolerance (BFT) consensus mechanism. Verified alerts are forwarded to the Collaborative Intrusion Detection System (CIDS) for further analysis. CIDS extracts features from each alert—including source and destination IPs, ports, protocol types, timestamps, TCP flags, and payload sizes—and applies min-max scaling to normalize them into structured vectors. These vectors are then processed by a trained Bidirectional Long Short-Term Memory (BiLSTM) model, which classifies each alert as either a malicious attack or abnormal network behavior.

IoT-Sentry [62] collects traffic packets in IoT networks by deploying sniffer nodes and converts the captured PCAP files into structured CSV format using the Wireshark tool. From these CSV files, the system extracts 21 cross-layer features, encompassing both packet-level and protocol-level attributes. These features are then standardized using Quantile Transformation and Min-Max Scaling. The normalized feature vectors are subsequently fed into a pre-trained ensemble learning-based Random Forest model to detect abnormal traffic. MalBoT-DRL [60] applies the Damped Incremental Statistics (DIS) method to process the traffic and extract 23 relevant features. These features are normalized using Min-Max scaling to form a standardized feature vector. The resulting vector is fed into a pre-trained Deep Q-Network (DQN) model, which combines Q-learning with a Deep Neural Network (DNN) architecture to enable intelligent traffic analysis and decision-making. Moreover, in [59], during the training phase, the authors select two kinds of datasets: a diverse cleaned dataset for labeling and an enhanced dataset for characterizing applications within diverse network traffic flows, in order to build a more representative training set. These two datasets are first merged into a unified structure using Python's Pandas and NumPy libraries. Then, the Scikit-learn library is used to identify and correct errors, inconsistencies, and inaccuracies in the data. Next, `MinMaxScaler` is applied to normalize all feature values into a fixed range to prepare for training. The processed vectors are used to train a model based on TBA and explainable artificial intelligence (XAI). For an unknown traffic instance, the system processes it using the same pipeline to generate a vector, which is then input into the trained model to predict whether the traffic is attack or benign.

In [63], the proposed method first preprocesses collected traffic packets using three techniques: Min-Max normalization, which scales numerical features to the [0, 1] range; mean imputation, which fills in missing values in numeric fields; and label encoding, which converts categorical fields (e.g., protocol type, TCP flags) into numerical format. From the preprocessed data, compact feature representations are then extracted using a stacked Contractive Sparse Autoencoder (CSAE). The compressed features are fed into a trained Bidirectional Long Short-Term Memory (BiLSTM) network, and the final classification result—whether the traffic is malicious or not—is produced by the network's Softmax output layer.

In [64], during the training phase, the system extracts 28 temporal and statistical features from multiple data sources, including ICS-SCADA and CIRA-CIC-DoHBrw-2020. These

features include phase voltage/current magnitudes and angles, sequence components, frequency variation, impedance, and relay states. A subset of low-correlation features is selected using the Pearson Correlation Coefficient (PCC), followed by Min-Max normalization, missing value imputation, and SMOTE oversampling to address class imbalance. This results in a dataset containing 499,672 samples. Each sample is represented as a 14-dimensional feature vector and fed into a CNN-LSTM model, where 1D convolutional layers (Conv1D) extract spatial features, and LSTM layers capture temporal dynamics. A fully connected layer and a softmax classifier are used to complete the model training. During the classification phase, the trained model is used to predict the class of new traffic samples, producing a three-class output: Non-DoH, Benign-DoH, or Malicious-DoH (including unknown zero-day attacks).

In [65], during the training phase, raw IIoT traffic data is preprocessed using one-hot encoding for categorical features and min-max normalization to scale numerical values. The processed feature vectors are then used to locally train a GRU-based model (comprising two GRU layers and one fully connected layer), which captures temporal dependencies in traffic sequences to identify abnormal patterns. Rather than sharing raw data, each device periodically uploads its model parameters to a central server, where a global model is updated using Federated Averaging (FedAvg). The updated global model is redistributed to edge devices. During the detection phase, new traffic data is processed using the same pipeline and passed through the locally deployed GRU model to classify whether the traffic is benign or malicious.

A line of research focuses on **automatically learning deep feature representations** from IoT traffic using either supervised or unsupervised learning techniques. Reconstruction-based approaches are widely adopted, where WADAC [57] leverages an Autoencoder and flags anomalies when the reconstruction error exceeds a threshold, while ADRIoT [66] employs an LSTM Autoencoder and evaluates reconstruction errors using RMSE. Similarly, Meidan et al. [46] combine Autoencoders with clustering (K-means) to separate benign and malicious behaviors, and Aljuhani et al. [63] propose a Contractive Sparse Autoencoder (CSAE) to generate robust feature embeddings. Beyond reconstruction-driven methods, Passban [67] applies one-class classifiers such as Isolation Forest and Local Outlier Factor (LOF) in a deviation-based framework, while FS3 [68] leverages self-supervised learning with TabNet and contrastive training for effective representation learning.

ADRIoT [66] and WADAC [57] employ different traffic collection and preprocessing strategies before feeding feature vectors into Autoencoder-based models. ADRIoT utilizes the *Scapy* tool to capture traffic packets, while WADAC leverages a TP-Link high-gain WiFi adapter and collects Zigbee traffic through Atmel RZ Raven USB sticks using the KillerBee toolset. After traffic acquisition, both systems process the collected packets into feature vectors with distinct methodologies. Specifically, ADRIoT transforms raw traffic

into fixed-length sequence vectors via three steps: (1) removing or masking irrelevant fields such as IP addresses and link-layer headers; (2) applying padding and normalization to unify sequence lengths; and (3) constructing time-series samples using a sliding window. In contrast, WADAC extracts both packet-level features (e.g., MAC addresses, frame types, flags, sizes, timestamps) and higher-level traffic features—namely Census, Ratio, Load, and Gap—from pcap files. The resulting feature vectors are then input into an LSTM Autoencoder or a standard Autoencoder. If the reconstruction error of a feature vector exceeds a predefined threshold, the corresponding traffic is flagged as anomalous.

All methods start from raw network traffic, whether in the form of packets, flows, or formatted results generated by tools such as NetMate and IPFIX. They then further process the collected traffic: [46] employs IPFIX enriched with statistical, DNS, and reputation-based features; [63] applies preprocessing steps including normalization, missing value imputation, and categorical encoding; [67] extracts statistical features (e.g., byte count, packet count, flow duration) from flows; and [68] transforms traffic into a tabular format consisting of numerical and categorical features. In the intermediate representation learning stage, both [46] and [63] rely on Autoencoder-based models: the former leverages an Autoencoder to learn reconstruction features of “normal” traffic, while the latter adopts a Contractive Sparse Autoencoder (CSAE) to extract compact representations. In contrast, [68] introduces a self-supervised learning model, TabNet, to capture dependencies among tabular features and map each sample into a latent vector space; whereas [67] directly models the distribution of “normal” traffic in the statistical feature space. Finally, in the anomaly detection stage, these methods primarily rely on deviations from normal traffic to identify malicious behavior: [46] employs Autoencoder reconstruction error combined with KMeans clustering; [63] uses a BiLSTM classifier on the compressed AE features; [67] adopts one-class classification models (iForest/LOF) to measure divergence from the normal distribution; and [68] performs detection using KNN enhanced by Few-shot and contrastive learning.

[69] and [70] both detect IoT botnets by analyzing DNS/domain-related features that may indicate command-and-control (C&C) connections. In the data collection stage, the two approaches focus on DNS traffic: [69] captures DNS packets from IoT networks, whereas [70] monitors DNS queries that return NXDOMAIN responses (which are likewise derived from DNS packets). In the data processing and detection stages, however, their methodologies diverge. Specifically, [69] converts collected DNS packets into human-readable logs, from which domain names are extracted. Each domain name is lowercased, mapped into character indices at the second-level domain using a constructed dictionary, and transformed into an indexed sequence. These sequences are embedded into vectors through the Keras embedding layer, padded to a fixed length, and first passed into a Siamese neural network that filters spoofed domain names. The remaining vectors are then processed by a pre-trained cost-sensitive deep learning

model (e.g., RNN or LSTM) to determine whether the domains were generated by a Domain Generation Algorithm (DGA), a technique commonly employed by botnets to establish C&C communications. By contrast, [70] applies a lightweight statistical approach. For each NXDOMAIN domain, it performs character-level analysis based on character frequency and bigram transition probabilities, and then applies a likelihood ratio test to construct a random walk model. As new NXDOMAIN domains are observed, the system incrementally updates a cumulative score; domains are classified as malicious if the score exceeds an upper threshold, and benign if it falls below a lower threshold.

Hardware feature: These studies primarily focus on leveraging different hardware and system-level features for malware detection. They collect a variety of features, such as hardware performance counters (HPC), embedded trace buffers (ETB), electromagnetic (EM) side-channel signals, system-level metrics, and power consumption traces. These features are then processed using various techniques like feature selection, dimensionality reduction, and time-series analysis to transform them into feature vectors or graph-based representations. The processed features are subsequently fed into machine learning models, including Decision Trees, Random Forests, Neural Networks, and Recurrent Neural Networks, to classify programs as either benign or malicious. The key idea behind these methods is to detect anomalies in system behavior, whether based on performance counters, communication patterns, or side-channel signals, which may indicate the presence of malware.

[71], [72], [73], [74], and [75] detect malware or anomalous behavior by collecting hardware performance counter (HPC) data or embedded trace buffer (ETB) outputs. In particular, [71] and [72], [73] both leverage HPCs to monitor run-time information such as Instructions Per Cycle (IPC), cache misses, and retired instructions. The approach in [71] employs three feature selection techniques—Univariate Selection, Feature Importance, and Correlation Matrix with Heat Map—to identify optimal subsets of HPC data. Based on these subsets, multiple classifiers, including Decision Tree (DT) and Neural Network (NN) models, are trained independently. During deployment, the best-performing combination of HPC features and classifiers from the training phase is selected for malware detection. In contrast, [72] focuses on constructing feature vectors from HPC-derived time series data using four techniques: Raw Samples, Context-switch Aggregation, Histograms, and Scheduling-based Aggregation. Each technique presents trade-offs in terms of effectiveness and applicability, and the optimal method is chosen through experimental evaluation under different scenarios. The resulting feature vectors are then classified using pre-trained models such as *K-Nearest Neighbors (KNN)*, *Decision Trees*, *Random Forests*, and *Artificial Neural Networks (ANNs)* to determine whether a given program exhibits malicious behavior. Similarly, given an unknown program, LAMBDA[73] executes it alongside a set of **predefined indicator programs**, such as `ls`, `ps`, and `netstat`, which respectively represent interactions with the

file system, process list, and network stack. During execution, the system uses `perf stat` to collect hardware performance counter (HPC) data. This process yields a matrix that captures the HPC behavior of the program under each indicator—e.g., CPU cycles, instructions retired, etc. LAMBDA then compares this matrix against a reference matrix built from known benign samples using both **Univariate T-Test** and **Multivariate T-Test**. If the distance metric from the first (univariate) test exceeds a predefined threshold, the program is classified as *malware*; otherwise, it is deemed *benign*. If the metric falls near the threshold, a second-round multivariate test is performed to compute a refined distance measure, which is then used to make the final classification decision.

During program execution, PREEMPT [74] utilizes a signal-selection tool, SigSET [76], to collect the lower 128 bits of the CPX (`cpx_spc_data_cx3`) as embedded trace buffer (ETB) signals. A sequence of signals generated over a period forms a trace. In the classifier training phase, data cleaning is performed by discarding traces where registers are reset to the default state, and by removing benign traces originating from benign functions within malicious programs. The cleaned signals are then converted into feature vectors, with four 32-bit vectors generated for each program. These vectors are used to train classifiers such as a K-Neighbors Classifier (KNN) or a Random Forest Classifier. In the detection phase, the trained model takes as input the feature vectors derived from ETB signals collected from unknown programs and determines whether the program is malicious.

In [75], the authors proposed a method that first collects hardware performance counters (HPC) and embedded trace buffer (ETB) outputs as hardware features. These features are then transformed into a two-dimensional representation of size $w \times d$, where w denotes the width (i.e., the number of traced signals), and d denotes the depth (i.e., the number of clock cycles observed by the trace buffer). During training, this 2D representation is used to train two models: a Decision Tree (DT) model and a Recurrent Neural Network (RNN) model with Long Short-Term Memory (LSTM) units. In the detection phase, newly collected hardware data is similarly converted into a $w \times d$ matrix and fed into either the DT or RNN model to predict the presence of malware.

Both [77] and ZeroD-fender [78] leverage power side-channel information for malware detection, although their approaches differ in scope and methodology. In [77], the authors use the Monsoon Power Monitor to collect power consumption traces from Android smartphones under benign and malicious conditions. Fast Independent Component Analysis (FastICA) is applied to extract independent signal components, and Recursive Feature Elimination (RFE) is employed to select the most informative features. These features are then used to train multiple classifiers, with Random Forest (RF) achieving the best performance for distinguishing benign and malicious traces. In contrast, ZeroD-fender targets zero-day malware detection in IoT devices by combining static code analysis with power side-channel data. Specifically, it extracts shared behavioral features (e.g., shell commands and

functions) through multi-scale static analysis, collects power traces from benign programs, and employs a multi-window LSTM autoencoder to select representative features. These features are subsequently used to train a deep learning model named ThingNetV2, which analyzes real-time power traces at deployment to detect previously unseen malware instances.

Unlike the aforementioned studies, EMMA [79] detects the presence of attacks by monitoring electromagnetic (EM) signals during the verifier–prover attestation process. The framework consists of three core components: a Verifier running on a trusted PC, a Prover implemented on an embedded device, and EM-Mon, a monitoring module responsible for capturing and analyzing EM side-channel signals. The attestation process starts with the Verifier generating a challenge, including a memory address range, the number of checksum iterations, a random initialization value, and a PRNG seed. This challenge is transmitted to the Prover, which triggers the execution of the attestation routine. During checksum computation, EM-Mon continuously monitors the EM signals emitted by the Prover and extracts three key features: (i) the delay between receiving the challenge and initiating the checksum process, (ii) the frequency of EM spikes during execution, and (iii) the total attestation time. Once the computation is completed, the Prover sends back a response containing the computed checksum and the original random value. The Verifier then compares the received checksum and nonce with the expected values and evaluates the extracted EM features against a known-good reference model. Any deviation—whether in checksum, nonce, or EM characteristics—leads EMMA to flag a potential attack or integrity violation.

Sentinel [80] uses system-level and process-level metrics for malware detection. It deploys a Sentinel Kernel Module (SKM) on each monitored device. This module collects both system-level and process-level metrics via the `sysfs` interface. These metrics include system-level information such as CPU frequency, memory usage, load averages, and the number of running processes, as well as process-specific statistics such as physical and virtual memory usage. All collected data is transmitted to a central hub via the MQTT protocol and stored in a PostgreSQL database. During the detection phase, Sentinel continuously queries the PostgreSQL database and feeds the retrieved data into a trained model for malicious behavior detection. Various machine learning techniques are supported, including Naive Bayes, rule-based learning (e.g., PART), logistic regression, multilayer perceptron (MLP), and tree-based models such as decision trees, random forests, and logistic model trees.

Runtime context: Recent studies on IoT and IIoT malware/attack detection leverage diverse types of features to capture device behavior and identify anomalies. Some approaches focus on **sensor and device data**, encoding raw readings or event sequences into structured representations for anomaly detection [81], [82], [83], [84], [85]. Others rely on **process- and system-level logs**, such as process behaviors, system calls, or attacker traces, to model normal execution and detect deviations [86], [87]. A third line of work exploits

communication signals, particularly in UAV networks, where physical-layer features such as Time Difference of Arrival (TDoA) and Received Signal Strength Difference (RSSD) are used to identify Sybil attacks [88]. Unlike the aforementioned approaches, the method in [89] analyzes the temporal dynamics of IoT devices for botnet detection. It first segments time into 100 ms intervals and counts the number of packets sent by each device within each interval, forming a time series. This time series is then transformed into dynamic behavior features using ordinal pattern analysis based on the Bandt and Pompe framework. Two types of features are extracted: probability distributions from the Ordinal Pattern Probability Distribution (OPPD) and graph-based features from the Ordinal Pattern Transition Graph (OPTG). After feature selection, the selected features are fed into a pre-trained Isolation Forest model. If the resulting anomaly score exceeds a predefined threshold, the device is flagged as being potentially infected by a botnet.

In [83] and [85], both methods adopt an *autoencoder-based approach*, but they are applied to different problem domains: malicious behavior detection and detecting false data injection (FDI) attacks in Industrial IoT (IIoT) systems, respectively. Specifically, in [83], the authors first train two autoencoders: one on sensor data under normal conditions and the other on sensor data during attacks. For an unknown traffic instance, both trained autoencoders are used to extract two sets of features from the sensor data, which are concatenated into a single super-vector. This super-vector is then reduced in dimensionality using Principal Component Analysis (PCA), yielding a compact feature representation. A One-Class SVM, trained exclusively on normal sensor data, evaluates whether the resulting feature vector deviates from the learned normal profile, thereby detecting unknown attacks. In parallel, the same feature vector is fed into a decision tree classifier, constructed using Gini impurity, to classify the instance as either benign or malicious.

Similarly, in [85], the system first collects raw data from 15 sensors on a hydraulic testbed to construct an input vector $\mathbf{z}$. To simulate attacks, a perturbation vector $\mathbf{a}$ is added to generate a tampered observation $\mathbf{z}_a = \mathbf{z} + \mathbf{a}$. An autoencoder (AE) is then trained using only benign data, where both the input and output are set to $\mathbf{z}$. By minimizing the reconstruction error

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (z_i - \hat{z}_i)^2,$$

the model learns correlations among sensor readings. During detection, if the reconstruction error for a new input $\mathbf{z}'_a$ exceeds a predefined threshold τ , it is classified as an FDI attack. To improve generalization and avoid overfitting, early stopping is applied during training. Furthermore, upon detecting an attack, the system employs a denoising autoencoder (DAE) to repair the corrupted input by mapping it back to the clean data space, thereby restoring the original structure.

[81], [82], and [84] follow a similar overall pipeline for malicious behavior detection. Specifically, each method first establishes a characteristic model of normal or legitimate

data during the training phase. In the detection phase, new observations are compared against this learned model, and any significant deviation is flagged as an anomaly. More specifically, PCoExtractor[81] first monitors interactions among IoT devices and encodes these behaviors into bit vectors: three bits are used to represent the current state, rising edge, and falling edge of binary sensors; another three bits represent the value range, trend (increasing/decreasing), and anomaly status of numerical sensors; one bit encodes the status of each actuator function. Based on these vectors, events are categorized into five types: Occurred, P-edge, N-edge, Uniform-1, and Uniform-0. During the training phase, the system filters out events that do not exhibit significant co-occurrence with the target event, and constructs a correlation and transition database representing valid event combinations and their temporal transitions. In the detection phase, PCoExtractor continuously collects event vectors and performs two checks: (1) whether the current combination of relevant events exists in the correlation database, and (2) whether the current transition exists in the transition database. If either check fails, the system reports an anomaly.

The method proposed in [82] focuses on analyzing sensor data from a forest fire monitoring network to distinguish whether it is normal, faulty, or under attack. The approach uses the Forest Fire dataset, which includes meteorological parameters collected by sensors (e.g., temperature, humidity, wind speed, and rainfall) along with each sensor's spatial coordinates (x, y). All data in the original dataset are considered legitimate. To simulate different types of anomalies, the authors manually generate labeled samples in three categories: *Legitimate*, using the original sensor readings; *Faulty*, by injecting Gaussian noise into individual sensors to simulate errors without spatial correlation; and *Attack*, by injecting similar noise into multiple adjacent sensors to preserve spatial correlation, mimicking coordinated attacks. A binary classification model—based on algorithms such as Classification and Regression Trees (CART), Random Forest (RF), or Support Vector Machine (SVM)—is then trained to distinguish normal from anomalous data. For samples classified as anomalous, Moran's I spatial autocorrelation index is applied to further determine whether the anomaly is due to a sensor fault or an attack.

Similarly, Chakraborty [84] proposes an FSA-based (Functional Subspace Alignment) method using the SWaT industrial water treatment system, where 54 IoT sensors collect water quality indicators (e.g., pH, chlorine concentration, conductivity) at a rate of one sample per second. The dataset includes seven days of normal operation and four days under attack scenarios. In the preprocessing phase, four key sensors are selected. The time series data are segmented into windows of 20–200 seconds, and each segment is smoothed using spline fitting to reduce noise. Bootstrap sampling and the Synthetic Minority Over-sampling Technique (SMOTE) are then applied to balance normal and attack samples. During feature extraction, each smoothed segment is treated as a functional curve, and its first derivative (velocity) is com-

puted. Functional Principal Component Analysis (FPCA) is performed on both the position and velocity curves to extract dynamic features such as level, slope, and peak values. These features are fed into multiple classifiers (M3–M6), which are trained and evaluated using Logistic Regression (LR), Support Vector Machine (SVM), Random Forest (RF), and *k*-Nearest Neighbors (KNN).

Both E-Spion [86] and AntiConcealer [87] are deployed on IoT devices or edge nodes to collect behavioral logs for malware detection. Specifically, E-Spion gathers three types of logs from benign executions: Process Whitelist Metrics (PWM), which records process names; Process Behavior Metrics (PBM), which capture execution time and resource usage; and System Call Behavior Metrics (SBM), which log sequences of system calls. To simulate malicious behavior, QEMU is used to execute malware samples and generate PBM and SBM logs. During training, PBM and SBM logs are transmitted to an edge server where two binary Random Forest classifiers are trained, while PWM is retained locally as a whitelist. In detection, any PBM or SBM logs not matching the whitelist are analyzed by the classifiers, and anomalies immediately trigger alerts.

In contrast, AntiConcealer deploys honeypots at edge nodes to capture attacker traces such as communication logs, SQL commands, and system logs, aiming to identify stealthy operations. The collected data are aggregated, filtered, encoded, split into training/testing sets, and time-normalized to preserve temporal order. For training, the system applies a Multivariate Hawkes Process (MHP) to model temporal dependencies between events and combines it with a Bayesian Probabilistic Graphical Model (BPGM) to construct an event influence graph. During detection, incoming data undergo the same preprocessing. The MHP first determines whether the activity is malicious; if so, the event is added to the propagation graph, and if its causal structure matches known attack patterns, it is flagged as a stealthy attack.

In [88], prior to model training, the authors construct a simulated UAV (Unmanned Aerial Vehicle) network in which some UAVs are programmed to launch Sybil attacks—i.e., transmitting signals under multiple forged identities—while others behave normally. Two detection nodes (denoted as x and y) are deployed to collect two types of signal information during UAV communications: the Time of Arrival (ToA) and Received Signal Strength (RSS). For each signal, the system computes the Time Difference of Arrival (TDoA) and the Received Signal Strength Difference (RSSD) between the two nodes. These computed TDoA and RSSD values are then labeled as either "Sybil" or "Genuine" and used to train four classifiers on the Weka platform: J48 (decision tree), Classification via Regression (a meta-learning method), OneR (one-rule classifier), and JRip (rule learner). During the testing phase, a similar simulated environment is used to generate new traffic data. The resulting TDoA and RSSD ratios are input into the trained models, which then predict whether the signal is from a Sybil or genuine source.

C2Miner[90] is a sandbox-based detection framework de-

signed to automatically execute IoT malware binaries and identify their Command and Control (C2) servers. The system takes two inputs: (a) a malware sample, and (b) a specified IP:port space of interest. Upon activation of the sample in the sandbox, the Man-in-the-Middle (MitM) module intercepts and redirects malicious connection attempts toward designated targets. For IP-based connections, redirection is achieved through direct substitution, while for DNS-based requests, the system modifies the local hosts file and resorts to fallback resolution if necessary. The profiler module then performs three key steps: filtering out irrelevant protocol traffic, recording access frequency to target addresses, and computing a C2 likelihood score for each destination. To further validate whether a target is a genuine C2 server, C2Miner supports two analysis strategies: (1) a SYN-DATA-aware approach that examines handshake and data exchange characteristics, and (2) a Fingerprinting-aware approach that constructs communication grammar models to extract behavioral fingerprints. Ultimately, C2Miner outputs (a) a list of active C2 server addresses, and (b) a set of communication fingerprint tags that support network forensics and attribution.

After being deployed in a smart home environment, Aegis+ [91] collects the daily activity data of various entities in the smart home, including sensors, devices, and controllers, and extracts trigger-action pairs from this data. The extracted pairs are then fed into a trained Markov Chain-based machine learning model, which is used to determine whether the observed behaviors are malicious. During the training phase, the authors first extract trigger-action pair rules from smart home apps using the data collector module. These rules are then used to generate a discrete context array that represents behavior patterns between sensors and devices. This context array is subsequently used to train the Markov Chain-based model, enabling it to learn and establish a baseline of normal behavior within the system.

Image: In existing dynamic analysis methods, the collected data—such as traffic, hardware performance counters (HPC), power signals or queries—is first transformed into image representations. These images are then either directly fed into pre-trained models such as CNNs, Vision Transformers (ViTs), or similarity-learning frameworks to determine whether malicious behavior occurs, or further processed by selecting more representative features, which are subsequently embedded into vectors and input into the model for malware detection.

[92] and [93] both propose methods that construct images from network traffic exchanged between IoT devices. The study in [92] collects forensic images derived from structured traffic logs, then removes features that may lead to overfitting, such as source IP, and applies Label Encoding to convert all categorical data into numerical vectors. Next, Min-Max Normalization is used to scale the feature values within a fixed range, after which Pearson Correlation is employed to eliminate highly redundant features. Finally, the Extra Trees (Extremely Randomized Trees) model selects the 13 most important features, resulting in the final feature vector. In contrast, [93] starts from raw network traffic and applies six

preprocessing steps: (1) segment the traffic into flows; (2) mask MAC/IP addresses; (3) remove empty and duplicate packets; (4) extract the first 784 bytes from each flow; (5) convert the byte stream into a 28×28 grayscale image using USTC-TK2016; and (6) transform the image into IDX format. The resulting IDX file is then fed into a pre-trained Vision Transformer (ViT) encoder to obtain a classification token, which serves as the final feature vector. In both approaches, the feature vectors are input into classifiers: a pre-trained Convolutional Neural Network (CNN) in [92] and an MLP classifier with a softmax output layer in [93], which produce the final benign-or-malicious classification.

LightAuditor [94], OD-meth [95], and [96] all follow a similar methodology in which side-channel features collected at the physical layer of program execution are transformed into image representations for further feature construction. The key differences lie in the type of side-channel data collected and the subsequent processing pipeline. In the dynamic information collection stage, LightAuditor [94] directly collects power consumption traces, while OD-meth [95] leverages a Raspberry Pi 3 equipped with an ACS712 current sensor to gather power consumption signals. During feature construction and vectorization, OD-meth [95] first transforms the collected one-dimensional power signals into two-dimensional time-frequency representations (TFRs) using Constant Q Spectral Transformation (CQT), and then applies Histograms of Oriented Gradients (HOG) to extract high-level structured features, resulting in HOG feature images. By contrast, LightAuditor [94] processes the collected traces through three steps: (1) applying a Root Mean Square (RMS) method to remove alternating current (AC) fluctuations, (2) using an alpha filter with a smoothing factor of 0.2 to suppress high-frequency noise, and (3) converting the filtered traces into Continuous Wavelet Transform (CWT) images. The resulting images are then fed directly into a trained Convolutional Neural Network (CNN), which classifies the program as either malicious or benign.

Unlike the above two approaches that rely on power consumption signals, [96] instead collects hardware performance counter (HPC) data rather than runtime energy consumption. Specifically, the QuickHPC tool is employed to record HPC events while the target application is executed in a sandbox environment. Principal Component Analysis (PCA) is then applied to select the most representative features from the monitored events. The selected HPC values, together with the application's binary content, are mapped according to functional attributes and uniformly converted into grayscale images. These images are subsequently fed into a Convolutional Neural Network (CNN) for final classification.

SQA [97] addresses malicious query detection in Industrial Internet of Things (IIoT) environments through a three-stage detection pipeline based on image transformation and similarity learning. First, the system extracts content from raw queries interacting with IIoT systems, converts the content into bytecode, and then transforms it into grayscale images to create a unified input format suitable for neural network pro-

cessing. Next, a similarity encoder built with a Convolutional Neural Network (CNN) maps these images into semantic embedding vectors. The encoder is trained using a contrastive loss function based on Mahalanobis distance, enabling the model to learn feature representations that accurately capture the similarity between queries. At runtime, the embedding of each new query is compared to historical queries using k-nearest neighbor (k-NN) matching. If its average distance to known malicious samples falls below a predefined threshold, the query is flagged as a potential attack and immediately blocked. Otherwise, it is passed to another CNN-based classifier for further assessment to determine whether it is malicious.

Spectral features: Existing studies focus on extracting spectral features from runtime signals such as electromagnetic (EM) emissions, raw power traces, or runtime memory snapshots. The extracted spectral features are then fed into classifiers to distinguish between malicious and benign programs.

The studies in [98], [99], and [100] all begin by collecting electromagnetic (EM) signals emitted during program execution. These signals are then transformed into a series of spectral samples (SS) using either Fast Fourier Transform (FFT) or Short-Time Fourier Transform (STFT). In [98], during the training phase, the collected SS are clustered based on their corresponding code regions using Density-Based Spatial Clustering of Applications with Noise (HDBSCAN), with the CAPE algorithm employed as the distance metric. After clustering, any new SS derived from incoming EM signals is considered anomalous and triggers an alert if more than 50% of its samples do not match any of the previously clustered regions. In [99], After using Fast Fourier Transform (FFT) to extract feature vectors from the collected EM signals, Principal Component Analysis (PCA) is used to reduce the dimensionality of the extracted features. These features are fed into a trained Random Forest or Support Vector Machine (SVM) model to detect the presence of malware. If malware is detected, the corresponding EM side-channel trace is segmented into multiple short windows. Each window is transformed into a spectrogram using Short-Time Fourier Transform (STFT). A high-dimensional feature vector is then extracted from each spectrogram using a combination of two-dimensional Discrete Wavelet Transform (2D-DWT) and mean pooling. Finally, after PCA-based dimensionality reduction, the resulting feature vector is input to a trained malware classification model (Random Forest or SVM) for malware type classification. EDDIE [100] collect EM emanations using a tool called *antenna*, and the signals are transformed into the frequency domain using Short-Time Fourier Transform (STFT), resulting in a sequence of Short-Term Spectra (STSs). For each STS, EDDIE extracts the peak frequencies, which serve as features representing the normal execution behavior of the program. In the detection phase, newly collected EM signals are processed in the same way—transformed into STSs and represented by their peak frequencies. The system then employs the Kolmogorov–Smirnov (K-S) test to compare the distribution of these frequencies with the reference distribution obtained during training. A significant divergence between

the two distributions indicates anomalous behavior, potentially caused by malicious activity.

In contrast to the aforementioned methods, which convert EM signals into spectral features for malware detection, *pAella* [101] and *DeepPower* [102] focus on another type of physical feature: voltage and raw power signals. More specifically, *pAella* [101] uses an Analog-to-Digital Converter (ADC) to continuously sample the current and voltage of the target device, storing the sampled data in designated memory. Once the number of samples reaches a predefined threshold, *pAella* applies the Welch method to compute the Power Spectral Density (PSD). The computed PSD is then written to RAM by the ARM processor. After accumulating several PSDs, the system performs Malware Detection (MD) inference locally using the NEON Single Instruction Multiple Data (SIMD) extension of the ARM architecture. In addition, a centralized monitoring unit communicates with the ARM processor via the Message Queuing Telemetry Transport (MQTT) protocol to dynamically determine which Machine Learning (ML) model to load and to identify the activities running in the Data Center (DC). Once malware is detected, the system promptly sends an alert to the DC’s system administrators.

DeepPower [102] continuously monitors IoT devices to collect raw power signals. A Robust Deep Autoencoder (RDA) model is first employed to remove normal components in the power signals, such as regular consumption patterns and background noise. The remaining signals are then further refined using a 100-point Simple Moving Average (SMA) filter and a wavelet denoising method. The filtered signals are transformed into mel-scaled spectrograms, which are passed through a Convolutional Neural Network (CNN) to extract deep features. These features are then fed into a Seq2Seq model based on Long Short-Term Memory (LSTM) networks to infer a sequence of device behaviors. Finally, *DeepPower* conducts a correlation analysis by comparing the inferred behavior sequence with a predefined infection process model, determining whether the device has been compromised by malware.

In Contrast, in [103], the authors explore a novel approach for detecting malicious behavior in IoT devices by converting runtime memory snapshots into audio signals and extracting features from those signals. Specifically, during the execution of an unknown program, the proposed system uses the *memfetch* tool to periodically collect memory snapshots for each process. These snapshots are concatenated into a binary stream, which is then mapped into a one-dimensional array using NumPy with a sampling rate of 48 kHz. The resulting signal is treated as a sound waveform. Next, a Short-Time Fourier Transform (STFT) is applied to generate the linear frequency spectrum. From this spectrum, three types of feature vectors are extracted using different methods: (1) Mel-Frequency Cepstral Coefficients (MFCCs) are computed by applying cepstral analysis; (2) Mel spectrograms are generated by mapping the signal to the Mel scale and applying magnitude normalization; and (3) Chroma STFT is derived by mapping frequencies to 12 pitch classes. In parallel, the

system performs a Constant-Q Transform (CQT) to obtain a Chroma CQT representation. This is further processed using energy normalization, temporal smoothing, and downsampling to produce Chroma Energy Normalized Statistics (Chroma CENS). All extracted features are then fed into a pre-trained Convolutional Neural Network (CNN) to classify whether the input memory snapshot contains malicious content.

Graph: Existing studies primarily extract graphs from three types of data for malware detection: network traffic, network temporal patterns, and data flows during program execution.

ContraMTD [104], GODIT [105], and Kalis2.0 [106] primarily collect network traffic between devices, and in some cases incorporate program context information, to construct a communication graph that characterizes the communication behavior of devices or applications. Subsequently, each method employs different technical approaches to analyze the graph for malicious behavior detection. More specifically, ContraMTD [104], GODIT [105], and Kalis2.0 [106] all follow a similar pipeline that starts from traffic collection and gradually transforms raw data into graph-based representations for malware detection. In data collection, ContraMTD collects benign raw traffic packets and extracts five-tuple information (source IP, destination IP, source port, destination port, and protocol), which are aggregated into flows, ordered into sessions, and further grouped into channels. GODIT takes traffic packets in .pcap format as input, while Kalis2.0 collects traffic via an I/O module and additionally gathers contextual features (e.g., from port scanning, banner grabbing, penetration testing, and traffic sniffing). After data collection, ContraMTD models the entire network as a host interaction multigraph, where nodes denote hosts and edges represent sessions. GODIT converts traffic packets into communication graphs, with nodes as device IPs and edges weighted by communication frequency. Kalis2.0 builds device profiles from collected features and constructs a Directed Acyclic Attack Graph (DAAG) in the cloud to capture potential attack paths. In vector embedding, for each channel, ContraMTD performs temporal segmentation, extracts statistical features, compresses them into vectors, and uses a **Fully Convolutional Neural Network (FCNN)** to obtain local feature vectors, while global features are extracted from the host interaction multigraph using **DE-GAT** (Graph Double Edge Attention Network). GODIT applies mixed-length random walks combined with **node2vec** to generate communication path embeddings, then selects the most discriminative features using **Shannon entropy**, constructing a 2048-dimensional vector. In Kalis2.0, device profiles and attack graphs serve as higher-level graph embeddings for context-aware analysis. After that, ContraMTD leverages **contrastive learning** to enforce consistency between local and global features, flagging traffic as malicious if this consistency is violated. GODIT feeds its 2048-dimensional vectors into a pre-trained **Robust Random Cut Forest (RRCF)** model to detect deviations from normal communication patterns, thereby identifying DoS attacks. Kalis2.0 deploys detection strategies from the cloud to edge nodes, where a hybrid engine combining **rule-based**

and **anomaly-based** modules detects malicious activities such as Telnet brute-force attacks, abnormal traffic, and suspicious control connections.

While current studies mainly utilize network communication between devices, [107] captures system behavior by constructing a system data flow graph for malware detection. In [107], the system first initializes a Quantitative Data Flow Graph (QDFG), which includes four types of nodes representing key system entities: Process, File, Socket, and Registry. Edges in the graph denote data flows between these entities and are annotated with attributes such as size and timestamp. During program execution, the system continuously monitors data flows by Windows API function call interposition, enabling real-time updates to the QDFG. In the malware detection phase, the system traverses the QDFG and applies three types of heuristic rules—replication heuristics, manipulation heuristics, and quantitative heuristics—to determine whether the observed system behavior matches known malicious patterns, thereby identifying potential malware.

In [108], nodes in an IoT network form a Destination Oriented Directed Acyclic Graph (DODAG) using the RPL routing protocol. A border router (BR) centrally collects and solidifies a four-dimensional behavioral baseline for each node, consisting of Time-To-Live (TTL), hop count, latency, and packet arrival rate. During runtime, each node compares the real-time values of these four features for every incoming packet against its baseline. If the TTL shows abrupt deviation or the latency/arrival rate exceeds predefined thresholds, a false-positive filtering mechanism is triggered—based on TTL differences and Trickle Consistency Notification (TCN) messages—to verify the presence of malicious traffic. Upon confirmation, the node broadcasts a lightweight alert message containing its rank. The BR then performs reverse rank-based analysis combined with sudden hop count changes to accurately locate both endpoints of a wormhole attack. These compromised nodes are blacklisted, and the DODAG is reconstructed, prompting all nodes to re-converge their four-dimensional baselines, thereby achieving a complete closed-loop detection and self-healing process.

2) *malware classification*: Three types of features are used for malware classification in existing approaches—**traffic, image, spectral feature**.

Traffic: MAFFIT [109] and LSTM-DNN [110],[111] do not directly treat the collected traffic as features or simply select features from existing datasets. More specifically, they adopt additional processing steps to derive higher-level representations before feeding them into the detection model. More specifically, MAFFIT [109] first constructs a sample set containing multiple labeled traffic samples from each class. During the attack detection phase, MAFFIT extracts two key sequences from each traffic instance to be analyzed: the packet length sequence (a sequence of packet sizes) and the packet byte sequence (a sequence of raw byte content). These sequences are then processed through two separate feature extractors: a Packet Byte Sequence Feature Extractor, and a Packet Length Sequence Feature Extractor based on

BiLSTM. The resulting feature vectors are fused into a single representation. The same process is applied to each sample in the reference set to obtain comparable feature vectors. MAFFIT then calculates the cosine similarity between the incoming sample and each reference sample. If the dissimilarity exceeds a predefined threshold, the traffic is classified as attack traffic. In contrast, LSTM-DNN [110] leverages real network traffic data from the N-BaIoT dataset for malicious behavior detection. For each unlabeled traffic sample, the system first performs data cleansing by removing missing values and infinite values. Then, Min-Max normalization is applied using a MinMaxScaler to scale the feature values to the [0,1] range, resulting in a normalized feature vector. This vector is then fed into a DNN-LSTM model that has been trained using 10-fold cross-validation. The model outputs a classification result indicating whether the traffic is benign or corresponds to a specific type of attack. Similarly, Sedjelmaci et al. [111] directly utilize flow-level records from the UNSW-NB15 dataset, where each record represents a network flow encoded as a 47-dimensional numerical feature vector. These features include statistical metrics (e.g., total source/destination bytes, packet count), behavioral indicators (e.g., number of forwarded/dropped packets, unique IP counts), and the protocol type. The resulting vectors are fed into a pre-trained detection model for attack classification.

Many other studies extract statistical features from traffic data [112][113], or employ hybrid feature representations that combine statistical and temporal characteristics for malicious traffic classification [114][115].

In [112], the authors propose *CorrAUC*, a hybrid feature selection algorithm that jointly considers the correlation between features and class labels (measured by the average Pearson correlation coefficient, avgcorr_{fc}) and the redundancy among features themselves (avgcorr_{ff}), for selecting features from network traffic data. The selected features are then ranked using an AUC (Area Under the Curve)-based wrapper method, and five representative statistical features are identified, such as Mean, Stddev, Ar_p_DdtIp, Pk_Src_IP, and Pk_Dst_IP. These features are then mapped into a soft set representation, and the optimal subset is determined using the Shannon Entropy TOPSIS method built upon a **bijective soft set** model. The selected optimal feature subset is finally fed into a trained classifier—such as the C4.5 decision tree, Random Forest (RF), Decision Table (DT), or Multi-Layer Perceptron (MLP)—to classify the collected network traffic according to the type of attack it represents. MENSA [113] first collects network traffic using Tshark and retrieves time-series data from SCADA systems via REST APIs. It then uses CICFlowMeter to extract flow-level statistical features such as packet count, byte count, inter-arrival time, and TCP flag indicators. These features are combined with SCADA measurements to form multimodal feature vectors. During the detection phase, the feature vectors are passed through a pre-trained Autoencoder-GAN. If the generator fails to reconstruct the input and the discriminator flags the behavior as highly abnormal—resulting in a significantly elevated adversarial

loss—the input is identified as anomalous. These anomalous vectors are then fed into a Conditional GAN (cGAN), which predicts the specific type of attack responsible for the abnormal traffic.

ACID [114] and Kundu et al. [115] both perform feature extraction and processing on network traffic to derive statistical and semantic representations, which are then aggregated into bidirectional flow-level features that characterize network behavior. More specifically, ACID [114] first constructs bidirectional flows from raw packets in the dataset and extracts two categories of features: (1) statistical features derived from packet headers, such as source/destination ports, inter-packet intervals, and packet counts; and (2) semantic features, optionally generated by applying Word2Vec and TextCNN to packet payloads, which are particularly useful for identifying content-based attacks such as SQL injection. These flow-level features are then processed by an adaptive clustering module that encodes them into low-dimensional embeddings and applies a kernel-based network to generate cluster centroids. The output of the clustering module is concatenated with the original features and passed to a Random Forest (RF) classifier, which performs the final traffic classification. In contrast, the system in [115] aggregates packets into unidirectional flows using the standard 5-tuple (source IP, source port, destination IP, destination port, and protocol), and then merges them into bidirectional sessions based on a predefined time threshold. From these flows and sessions, the system derives a rich set of features, including aggregated features (e.g., sum, count, min, max), temporal features (e.g., session duration statistics), statistical features (e.g., mean and standard deviation of packet/byte counts), and connection-context features that capture multi-flow correlations. The resulting feature vectors are then fed into a one-dimensional Convolutional Neural Network (1D-CNN), which not only detects the presence of botnet-related activity but also classifies the specific type of malicious behavior.

Some studies further refine the initially extracted features by applying deep neural architectures such as CNNs, BiLSTMs, and LSTMs to capture richer contextual, temporal, and spatial dependencies [116], [117], [118], [119]. Other approaches focus on representation learning through autoencoders, where latent feature vectors are obtained by minimizing the reconstruction error; these vectors are then used for malicious traffic or attack classification [120], [121].

In [116], for test-time traffic from ISCXPVPN-nonVPN and USTC-TFC2016 datasets, different preprocessing strategies are applied. For the former, each TCP/UDP packet is padded or truncated to 1500 bytes, then normalized to [0,1] by dividing by 255. For the latter, traffic is represented using Flow + L7 structure, with randomized MAC/IP fields and fixed-length 784-byte packets similarly normalized. These vectors are input to a lightweight CNN based on the Network-in-Network (NIN) architecture, where 1×1 convolutions and max pooling are used for local modeling and dimensionality reduction. The resulting feature vectors are fed into a trained classifier (trained by two-stage knowledge distillation) with a softmax output

layer, yielding one of 20 categories (e.g., AIM, ICQ). In [117], the authors perform a three-step preprocessing pipeline to transform raw traffic into vectorized input. First, static identifiers such as IP addresses, ports, timestamps, and flow IDs are removed; second, records containing NaN or null values are discarded; third, all numerical features are scaled to the range [0,1] using Min-Max normalization. The processed feature vectors are then fed into a hybrid deep learning model comprising a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) module. The CNN captures spatial dependencies among features within individual traffic samples, while the LSTM learns temporal patterns from the sequentially ordered features. The outputs of both modules are concatenated into a unified representation, which is passed through fully connected layers and a softmax classifier to predict whether the traffic is benign or belongs to a specific type of DDoS attack.

LKD-STNN [118] processes raw network traffic by first segmenting it into flows based on 5-tuples. For each flow, the first n packets are retained, with each packet truncated or zero-padded to a fixed length m , byte-encoded into values within [0, 255], and normalized to the [0, 1] range. This results in a fixed-dimensional vector sequence that preserves both temporal structure and payload content. During detection, the teacher model leverages a 1D-CNN to extract spatial statistical patterns and a BiLSTM to capture sequential dependencies from the fixed-dimensional vector to produce soft labels. The student model adopts a lightweight architecture based on 1D depthwise separable convolutions and a simplified BiLSTM to learn both hard ground-truth labels and the teacher's soft outputs. By jointly modeling spatial and temporal patterns, the final model enables accurate detection of malicious traffic such as DDoS, backdoor, and command-and-control (C&C) communications.

CMFTC [119] begins by parallelly extracting two types of modal information from encrypted traffic: the payload and the protocol header. The payload is first processed with bidirectional flow-relative position encoding and then passed through a custom-designed Cross Block, yielding two outputs: a low-dimensional semantic vector (PS_i) and a high-dimensional feature matrix (PL_i). Concurrently, the header is used to extract statistical features such as payload length, TCP window size, inter-packet intervals, and directionality. These features are concatenated with the corresponding PS_i to form an enriched vector sequence. This sequence is then processed by a multi-layer 1D depthwise convolution module to extract contextual features (C_i), capturing local temporal dependencies. The resulting contextual sequence is interleaved with the high-dimensional payload matrix PL_i to construct a unified global, flow-level semantic representation. During the malware detection and classification phase, the final flow-level semantic vectors are passed into a shared representation layer. This is followed by multiple softmax output heads for multi-task learning, including T1 (binary classification: benign vs. malicious) and T2 (fine-grained malware categorization).

Unlike the aforementioned methods, Aouedi et al. [120]

adopt a federated learning framework to train an autoencoder for feature extraction and vectorization. In the detection phase, unlabeled samples from benchmark industrial datasets—each consisting of 23- or 26-dimensional features that capture various aspects of network traffic and control system behavior (e.g., pressure, valve state, flow rate, tank level, pump state, command signals, and sensor readings)—are input to the trained autoencoder. The encoder maps each sample into a low-dimensional latent vector, which is subsequently reconstructed back to the original feature space. By minimizing reconstruction error, the autoencoder is optimized to learn informative representations. These latent vectors are then passed to a fully connected neural network (FCN) for classification, yielding binary decisions on whether the input represents benign or malicious activity.

MCRc [121], by contrast, targets noisy-labeled network traffic samples and employs a 7-layer fully connected encoder to derive low-dimensional vector representations. To construct a semantically consistent and noise-resilient embedding space, MCRc introduces three joint constraints during encoder training: (1) Information Preservation Constraint — a symmetric decoder G^{-1} forms an autoencoder structure, minimizing binary cross-entropy between input and reconstructed features to preserve traffic characteristics; (2) Inter-Class Separability Constraint — the normalized margin $\delta = \frac{d_{out} - d_{in}}{d_{out}}$ is maximized, where d_{in} denotes intra-class mean distance and d_{out} the minimum inter-class distance, thereby enhancing class discrimination; and (3) Intra-Class Compactness Constraint — high-confidence samples (identified via softmax confidence) are selected as class centers z_k , with other samples optimized to cluster around them. The resulting embeddings are classified via KMeans, and clustering results are aligned with ground-truth labels using the Kuhn-Munkres matching algorithm, enabling robust categorization of traffic into classes such as DDoS, Botnet, Backdoor, or Benign.

Image: Existing image-based approaches for malicious traffic classification generally follow a similar pipeline. Raw traffic captured in .pcap format is first transformed into fixed-length byte sequences (e.g., 784 bytes), which are then converted into images by using tools like USTC toolset or IDX representations. Among these, some studies adopt grayscale images [122], [123], [124], [125], while others employ RGB images [126]. Deep neural networks are subsequently leveraged to extract semantic representations, and the final classification is performed using either a softmax layer or a separate classifier such as logistic regression (LR).

More specifically, in the traffic-to-image transformation stage, studies such as [122], [123], [124], [125] generally adopt a multi-step preprocessing pipeline that includes traffic segmentation, cleaning, image generation, and IDX file construction, although the details vary slightly across works (e.g., session truncation in [125], or application-layer slicing in [124]). In the feature learning stage, most methods employ convolutional neural networks (CNNs) to capture deep spatial and semantic patterns from traffic images. Some studies further enhance CNN-based representations with lightweight

architectures [125], neural architecture search [123], or semi-supervised/domain-adaptive components [124]. In particular, Ning’s framework [124] augments CNNs with a Ladder Network to exploit both labeled and unlabeled data, incorporates feature-level domain adaptation for cross-domain robustness, and extends shallow CNNs to deeper VGG-based variants (KTDA-VGG and KTDA-VGGLadderNet), which significantly improve classification performance. Finally, the learned feature vectors are fed into either a softmax classifier or, in the case of [122], a logistic regression (LR) model to produce the final traffic category.

Similarly, in [126], the IoT edge gateway first collects malicious network traffic packets (.pcap files), which are then preprocessed through four steps: traffic segmentation, data cleaning, traffic anonymization, and removal of duplicate data. Each pcap file is subsequently converted into an RGB image using the USTC toolset. These images are then input into a trained DSAN-AT model, which uses a softmax function to predict the probability distribution over traffic classes. The DSAN-AT model mainly consists of two key components: the LMMD module (Local Maximum Mean Discrepancy), which enhances classification performance on previously unseen malicious traffic, and the CBAM module (Convolutional Block Attention Mechanism), which focuses on more informative regions or byte-level features within the input.

spectral features: In [127] and [128], the target programs are executed inside a sandbox, where side-channel signals are collected during runtime. Specifically, [127] uses GPIO pins to trigger an oscilloscope for capturing electromagnetic (EM) emissions, while [128] records power consumption signals via an onboard ADC at a sampling rate of 2 million samples per second. In both cases, the collected time-domain signals are transformed into time–frequency spectrograms using the Short-Time Fourier Transform (STFT). In [127], Normalized Inter-Class Variance (NICV) is further applied to select the most discriminative frequency bands within the spectrograms, and the resulting features are fed into a Multi-Layer Perceptron (MLP) and a lightweight Convolutional Neural Network (CNN) to predict the malware type. By contrast, [128] directly inputs the spectrogram vectors into a supervised Support Vector Machine (SVM) classifier to distinguish between benign execution and different categories of injected attacks.

3) *Lessons learned:* We survey the existing dynamic feature extraction techniques and analysis methods for IoT malware detection and classification. The following highlights the key lessons learned from this investigation.

- Extracting or selecting features from network traffic for malicious behavior detection and classification has become the most widely adopted research direction. Some studies directly select specific fields from processed traffic features in public datasets, or capture raw pcap files using tools such as Wireshark, Tcpdump, and Sniffer-Pro, followed by preprocessing techniques like Min-Max normalization, standardization, and one-hot encoding to generate model-ready input vectors. Other approaches employ tools such as CICFlowMeter, Bro/Zeek, IPFIX,

and NetMate to extract statistical traffic features. Temporal sequence features are also widely adopted and have been shown to be effective in IoT attack detection. Furthermore, deep learning–based representation methods, including CNN, BiLSTM, CNN+LSTM, Word2Vec, and TextCNN, have been used to learn richer feature vectors, which are subsequently fed into classifiers for detection or classification. Overall, researchers have explored diverse feature extraction and selection methods to derive more effective representations from network traffic. A comparative summary of traffic feature extraction techniques is shown in Table IV.

- Hardware information and runtime context features of IoT systems are highly robust, as they can be directly observed by specific devices and accurately capture the overall system behavior. These features are drawn from diverse sources: hardware information includes hardware performance counters (HPC), embedded trace buffers (ETB), power traces, and electromagnetic (EM) side-channel signals; runtime context information includes sensor data, system or process logs, system call sequences, and communication signals (e.g., TDoA, RSSD). Some studies embed these features directly into vector spaces after noise reduction, while others model them through temporal logic or statistical methods to derive behavioral representations. More recent work explores visualization-based representations, such as mapping hardware information into CWT, CQT, or HOG images; applying STFT, FFT, PSD, or CQT to generate time–frequency spectral features; or modeling system behavior as data flow graphs. These approaches aim to investigate whether such representations enable neural networks to learn more effective behavioral features. A comparative summary of hardware and runtime context feature representations is provided in Table V.
- In summary, traffic features have been extensively studied in terms of extraction and processing techniques, while visualization-based representations of hardware and runtime context features remain an emerging research direction. However, two critical challenges persist: (1) hardware information often contains substantial noise, and developing effective noise reduction techniques while preserving discriminative characteristics remains difficult; and (2) the collection of hardware and context features typically requires deploying specialized programs or devices on the target IoT system, which introduces resource overhead. Reducing such overhead while maintaining detection accuracy is an important challenge for future research.

C. Hybrid Analysis Features

In existing research, many studies rely on a combined use of static and dynamic analysis to detect malware in IoT environments. A general workflow of hybrid analysis is shown in Figure 4. Typically, static analysis is first applied to parse

TABLE IV
TRAFFIC FEATURES FOR MALWARE ANALYSIS

Paper	Feature Engineering	Performance
Detection		
[37]	dataset	99.99%, 99.99%, 99.99%
[38]	feature pruning	87%
[39]	traffic parsing	82.9%, 88.0%, 82.9%, 82.3%
[40]	traffic parsing	93.5%, 97.5%, 90.5%, 93.5%
[41]	traffic parsing, PCC	94.56%, 94.94%, 95.58%, 94.56%
[42]	ipfixprobe, traffic parsing	99%, 100%, 94%, 97%
[47]	CICFlowMeter,XGBoost/SULOV	92.6%
[43]	tcpdump, statistic	93.6%, 94.5%
[44]	traffic simulation,temporal analysis	84%, 85%
[45]	NMF	91.4%
[46]	dataset,aggregation,vectorization	84.1%, 92.9%, 100%, 1.4%
[48]	tcpdump,Bro-IDS,customized extractor	98.8%, 97.8%, 2.0%
[49]	tcpdump, domain name analysis	94%, 96%, and 96%
[50]	OpenFlow switch	95.7%, 95.7%, 95.4%, 95.6%
[51]	-	46% – 87%
[52]	traffic parsing	95.6%
[53]	CHT,FIM	97.7%, 97.7%
[54]	traffic parsing,EV calculation	84.5%, 2.8%
[58]	Kitsune,PCA	94.2%, 94.3%, 93.9%, 94.1%
[56]	tcpdump,Probing,tsfresh	94.3%, 5.7%
[55]	traffic parsing	99.7%, 99.7%, 99.7%, 99.7%
[57]	scapy,traffic parsing,statistic	97.46%, 86.8%, 99.5%
[66]	Scapy,vectorization	93.7%
[67]	NetMate	96.4%, 90.2%, 93.1%
[68]	dataset,vectorization	81.3%, 76.6%, 76.3%
[69]	traffic parsing,vectorization,statistic	98.0%, 98.0%, 98.0%, 98.0%
[70]	DNS traffic parsing,statistic	100%, 94%, 11.5 NXDOMAIN, 0.24%, 99.8%
[61]	dataset, vectorization	99.16%
[62]	traffic parsing,random forest	99.5%, 99.9%, 99.9%, 99%
[63]	traffic parsing,CSAE	99.3%, 99.2%, 99.4%, 99.3%
[64]	dataset,PCC,vectorization	of 99.3%, 99.3%, 99.8%, 99.8%
[65]	dataset,vectorization	99.3%, 98.1%, 98.7%, 98.6%
[59]	CICFlowMeter,PyShark,Scikit-learn	99.4%, 99.3%, 99.4%, 99.3%
[60]	Kitsune	99.6%, 99.9%, 99.9%, 99.9%
Classification		
[109]	traffic parsing	99.0%, 99.0%, 99.0%, 99.0%
[110]	traffic parsing,vectorization	99.9%, 99.9%, 99.9%, 99.9%
[111]	dataset,vectorization	96%, 7%
[112]	dataset,Correlation analysis	99.9%, 99.8%, 99.8%, 99.5%
[113]	Tshark,REST API,vectorization	94.7%, 81.2%, 3.6% and 79.4%
[114]	traffic parsing,Word2Vec/TextCNN	100%, 100%
[115]	traffic parsing,vectorization	94.3%, 98.7%, 94.2%, 99.7%, 98.6%
[116]	traffic parsing,vectorization	95.8%, 95.76%
[117]	traffic parsing,vectorization	99.4%
[118]	Wireshark,SplitCap,Scapy	98.6%, 98.1%, 98.2%, 98.0%
[119]	Scapy,traffic parsing	97.72%, 94.68%
[120]	dataset,vectorization	98.8%, 98.5%
[121]	traffic parsing,vectorization	82%, 85%

raw binaries, source code, or bytecode, extracting contextual information such as API calls, abstract syntax trees (ASTs), or control flow graphs (CFGs). In some approaches, these static artifacts are directly compared with those of benign programs using techniques such as pattern matching to identify potential malware. Once suspicious behavior is detected, the program may then be executed and monitored at runtime to improve detection accuracy. Other approaches instead use the static information to guide dynamic execution within controlled environments such as sandboxes or Genymotion emulators. Dynamic behaviors are then captured through runtime monitoring or binary instrumentation, and together with static features,

they are analyzed using methods such as temporal analysis or state transition analysis. Ultimately, static and dynamic features may also be fused into a hybrid representation, which is then fed into a trained machine learning–based model for malware detection.

1) *malware detection*: **Context-sensitive feature**: Existing studies adopt different strategies to combine static and dynamic program information for malicious behavior detection. Some approaches rely on static analysis (e.g., using tools such as AstBuilder) to parse source code or APK files and extract the expected behavior patterns of benign programs. During detection, the program is dynamically executed and

TABLE V
DYNAMIC-ANALYSIS FEATURE FAMILIES, REPRESENTATIVE PAPERS, AND CONCISE FEATURE ENGINEERING

Paper	Feature Engineering	Performance
Image		
[122]	USTC-TL2016 toolkit	96.8%
[123]	B2P Mapping	99.9%
[124]	traffic processing,B2P Mapping	99.2%
[125]	traffic processing,B2P Mapping	99.9%, 99.5%, 99.4%, 99.4%
[126]	USTC toolset	97.1%
[92]	traffic parsing, vectorization	81.7%, 81.7%, 81.7%, 81.7%
[93]	traffic processing,B2P Mapping	–
[94]	CWT	–
[95]	CQT, HOG	–
[96]	QuickHPC, B2P Mapping	–
[97]	B2P Mapping	–
Spectral		
[98]	FFT, STFT	–
[99]	FFT, STFT, DWT	–
[100]	STFT	–
[101]	PSD with Welch	–
[102]	FFT	–
[103]	STFE, acoustic encoding	–
[127]	STFT, NICV	–
[128]	STFT	–
Hardware		
[71]	HPC, Linux perf,vectorization	–
[72]	HPC, runtime monitoring	–
[73]	HPC, Linux perf	–
[74]	ETB,SigSET,vectorization	–
[75]	HPC,ETB,vectorization	–
[77]	FastICA, RFE	–
[78]	correlation analysis	–
[79]	electromagnetic side-channel analysis	–
[80]	Linux tool,vectorization	–
Runtime Context		
[83]	dataset,vectorization,PCA	–
[85]	correlation analysis,vectorization	–
[81]	PCoExtractor Compiler	–
[82]	dataset,vectorization	–
[84]	FPCA,FSA	–
[86]	runtime monitoring,vectorization	–
[87]	runtime monitoring,vectorization	–
[88]	runtime monitoring,math calculation	–
[89]	ordinal pattern analysis	–
[90]	traffic parsing, C2 Analysis	–
[91]	Markov Chain	–
Graph		
[104]	traffic parsing,graph analysis	–
[105]	node2vec,random walk, graph sketching	–
[106]	traffic parsing,statistic	–
[107]	data flow analysis	–
[108]	graph analysis	–

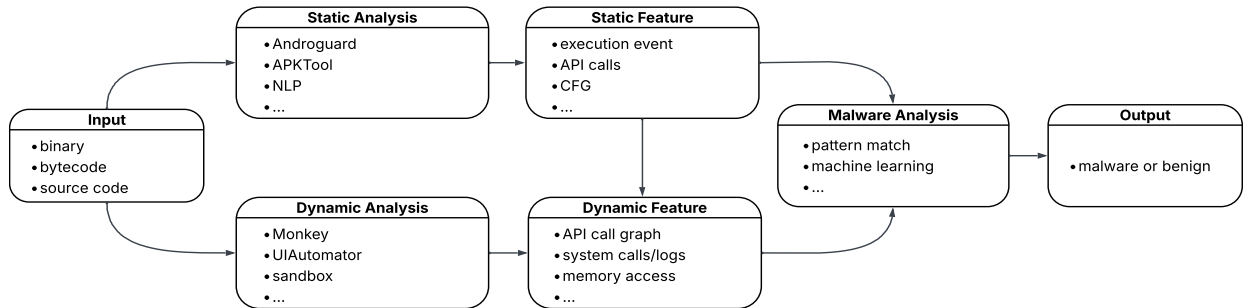


Fig. 4. Workflow of hybrid analysis.

the observed runtime behavior is compared against the pre-extracted patterns to identify potential malicious actions [129], [130], [131]. Other studies directly integrate features obtained from both static and dynamic analyses to train a unified model for inferring malicious behaviors [132], [133], [134], [135]. More advanced methods perform a two-step process: first, static features are used for an initial classification to determine whether a program is suspicious, and then dynamic features are employed either to further evaluate the severity of the maliciousness [136] or to re-verify the classification decision for improved accuracy [137].

For each SmartApp under inspection, Aegis[129] performs static analysis of its source code to extract the trigger-action logic, which is formalized as the *App Context*. The collected runtime data and the App Context are then combined to construct a *context array*, representing the runtime correlations among program states, sensors, and devices. Each context array is processed by a Markov Chain model to compute state transition probabilities; transitions with extremely low probability are flagged as anomalous or malicious behaviors. Moreover, given a pre-trained transition model derived from benign devices and applications, Aegis can incorporate user-confirmed benign transitions during detection, thereby expanding and refining the model. HoMonit[131] first derives the expected deterministic finite automata (DFAs) of benign SmartApps. For open-source SmartApps, the *AstBuilder* tool is used to convert the source code into an abstract syntax tree (AST), which is then translated into a DFA. For closed-source SmartApps, natural language processing (NLP) techniques are applied to parse the textual description into a DFA. These DFAs capture the expected program behavior, including initial states and valid state transitions during execution. In the detection phase, HoMonit collects encrypted traffic between the hub and smart devices using different sniffing tools (e.g., CC2531 for ZigBee and USRPs for Z-Wave). After filtering out beacons and irrelevant packets, the remaining payloads are segmented into bursts and translated into device event sequences. These sequences are compared against the expected DFA; mismatches—such as the occurrence of unauthorized commands or attributes (absent in the DFA) or skipped mandatory transitions—are flagged as potential injections or spoofed events, indicating an attack.

In contrast, [130] focuses on combining developer-provided system requirements with program binaries to extract static features and construct a Runtime Security Model (RSM). In this approach, program execution events are categorized into four types: branch events, triggering events, regular events, and pair events, and are further semantically classified into *events*, *control flow events*, *nullified control flow events*, and *mimicry malware sequences*. From the system requirements, the authors extract the Best-Case Execution Time (BCET) and Worst-Case Execution Time (WCET) of each event, along with their execution order and temporal relationships, to build a complete RSM. To improve practicality, three greedy algorithms are proposed to select the most representative RSM* subsets based on time range, time variance, and peak probability,

which are then deployed in hardware for runtime monitoring. During execution, the RAD hardware collects real-time event sequences and timing information through the CPU trace port and applies three detection procedures—sequence check, timing check, and nullified event check—to determine the presence of malicious behavior.

In the process of program behavior representation, combining dynamic and static features can provide a more comprehensive abstraction of program behavior. In [132], the training phase integrates dynamic features (e.g., system logs, program execution traces) with static features (e.g., API calls) to build the detection model. A honeypot system is deployed to collect behavioral data from both ransomware and benign software, including source programs. After data cleaning, two categories of features are extracted: signature-based features (SigRW), such as hashes and rules, and static vector features (FTRW), such as code structure and API calls. These features are then used to train several traditional machine learning models, including Random Forest, Logistic Regression, and Decision Tree. During the detection phase, the honeypot system monitors runtime behaviors such as API calls, which are processed into feature vectors using the same feature extraction procedure, and these vectors are subsequently fed into the trained models to determine whether a sample is ransomware.

In [133] and [134], the authors leverage both static and dynamic analyses to characterize Android applications. Specifically, [133] employs *Androguard* to decompile APK files and extract static features such as permissions and API calls, whereas [134] uses *APKTool* to decompile APKs and extract the *AndroidManifest.xml*, *.dex* files, and resource files. From these artifacts, regular expression matching is applied to retrieve API calls, permissions (e.g., *INTERNET*, *BIND_DEVICE_ADMIN*, *READ_PHONE_STATE*), and embedded threat-related text, which are then used to construct static feature vectors. To capture dynamic behaviors, both studies execute the applications in a *Genymotion* emulator, where the *monkey* tool is used to generate random events and stimulate program execution. A key distinction is that [134] additionally captures runtime screenshots and applies OCR techniques to identify threatening messages or dialogs; the extracted text is then combined with text from resource files to build dynamic feature vectors. After obtaining both static and dynamic features, these vectors can either be directly concatenated into a unified representation and fed into classifiers such as Support Vector Machine (SVM), Random Forest (RF), or Naïve Bayes (NB), or further preprocessed through steps such as text tokenization, noise removal, and image normalization (e.g., scaling, grayscale) to ensure compatibility with classification models. Finally, the processed features are input into pre-trained classifiers—including SVM, Decision Tree, RF, NB, and Logistic Regression—to determine whether the application is ransomware or benign software.

Furthermore, Mini-Me[135] targets the detection of data-oriented attacks on robotic aerial vehicle (RAV) control programs in IR-level through a three-stage process: control program instrumentation, approximate model construction,

and online detection. First, the control program's binary is converted to LLVM IR to generate a control flow graph (CFG), and symbolic taint analysis is applied to locate target mathematical functions; dynamic binary instrumentation (e.g., Valgrind) records their call frequencies, while basic blocks are recursively traversed to count arithmetic instructions. Static data-flow analysis then extracts critical state variables, identifying the input set (used-before-defined) and output set (newly defined variables), while accounting for intermediate variables, array accesses, and pointer aliases. These variable sets define the input/output nodes of an LSTM model, ensuring semantically interpretable features. In a secure environment, normal execution data is collected, normalized, and segmented with a sliding window to produce temporal samples for training a lightweight LSTM that learns the input-output sequence mapping, optimized via mean squared error and Adam, followed by quantization and compression for deployment. During online detection, Mini-Me injects monitoring functions into the control loop, predicts outputs from real-time inputs, and flags compromise when prediction errors exceed a confidence interval derived from benign data, triggering fail-safe protection.

ITEC [136] first extracts a runtime signature from the application, which includes behavioral features such as system calls (e.g., `NtCreateProcess`, `NtCreateThread`, `NtListenPort`), memory access patterns, and network connection behaviors. This signature is then compared against a pre-identification database containing known malware and benign application signatures. If the result is suspicious, the application is executed in a sandbox environment, and a threat score is computed by aggregating scores across three dimensions: exploitability, impact, and environment. Based on this threat score, the application is classified into low, medium, or high risk. Similar to these approaches, [137] first uses Apktool to decompile the APK, obtaining `AndroidManifest.xml` and `.dex` files. From the `.dex` file, the Androidguard extractor module extracts API requests and network-related information (IP addresses, email addresses, URLs). From `AndroidManifest.xml`, it extracts intents and permissions. These features are stored in a text file, which is then converted into a fixed-dimensional numeric vector. This vector is fed into a pre-trained classifier to predict whether the APK is malicious or benign. If classified as benign, the system proceeds to dynamic analysis. The APK is executed in an Android emulator, and user interactions are simulated using Monkey or UIAutomator to trigger events. An API call graph is then constructed, where each system call is represented as a node, node size indicates the call frequency, and directed edges represent the execution order of system calls. This graph is processed by four heterogeneous sub-models—each focusing on a specific feature set (Permission, Intent, API, Hardware). The outputs of these sub-models are combined to produce the final classification result.

2) Lessons learned:

From recent studies on hybrid-analysis-based features for

malware detection and classification, we learn the following key lessons:

- Static features can reveal a program's potential runtime behaviors, but their effectiveness may be undermined by code obfuscation. In contrast, dynamic features capture the actual behavior during execution and enable real-time detection, but they are costly to collect and often fail to cover all possible execution paths. Combining static and dynamic features helps improve detection accuracy and coverage. Existing studies mainly adopt three hybrid strategies. The first extracts expected behaviors of benign programs through static analysis (e.g., trigger-action logic, DFA), and then compares them with the observed runtime behaviors of the target program to detect deviations that may indicate malware. The second integrates static and dynamic features into a unified representation, which is then fed into ML/DL models for classification. The third employs a two-stage process where static analysis performs an initial coarse screening, followed by dynamic analysis to refine or validate the detection results.
- Hybrid context-sensitive features combine static contextual information, such as APIs, permissions, and textual attributes, with dynamic contextual information, such as runtime logs and execution traces. Existing studies mainly leverage these hybrid features for malware detection, while their application to malware classification has received little attention. Future research could explore alternative combinations, such as integrating image-based features with dynamic contextual information, to enhance both malware detection and classification.

D. Graph Learning Representation Features

With the rapid advancement of machine learning and deep learning techniques, an increasing number of studies have explored the use of *graph-based learning approaches* for IoT malware detection and classification. In these approaches, the analyzed binaries or bytecode are typically transformed into program behavior graphs such as *call graphs* or *control flow graphs (CFGs)*, while in other cases, device-to-device communication is modeled as a *traffic-based graph*. In the subsequent stage, these graphs are converted into vector representations: one line of research leverages deep neural networks to perform *graph embedding*, mapping the entire graph structure into vectors, whereas another line of research relies on *feature extraction and selection techniques* to compute statistical or structural properties from the graph and construct feature vectors. Finally, during the detection or classification phase, the resulting vectors are fed into trained models to identify or classify IoT malware samples. The workflow of the graph learning representation analysis is shown in Figure 5.

1) *malware detection*: Existing studies employ a variety of tools to extract different types of graphs for representing program behaviors. For example, some approaches use ApkTool to extract APIs along with their corresponding code block IDs, which are then used to construct API graphs [138].

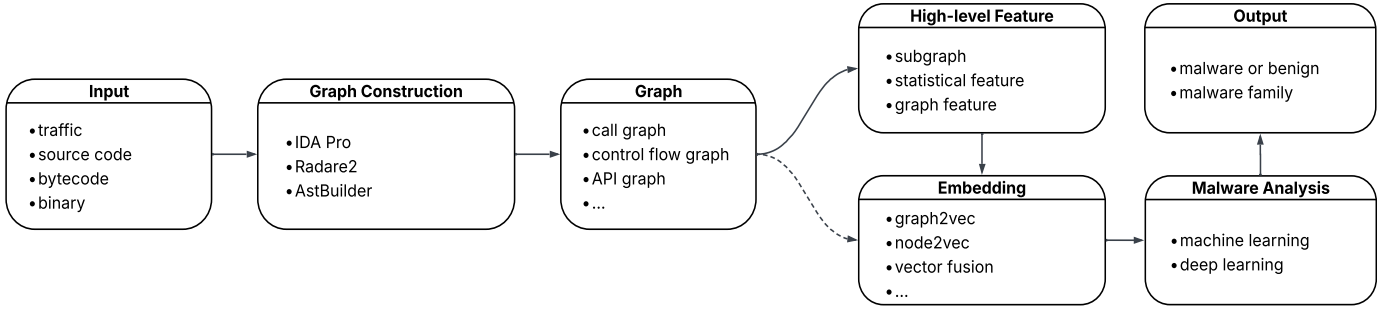


Fig. 5. Workflow of Graph learning method.

Others leverage IDA Pro to extract function call graphs (FCGs) from binaries and subsequently derive graph-based features to characterize program behavior [139]. Similarly, Androguard, in combination with the visualization tool Gephi, has been utilized to generate FCGs from Dalvik bytecode [140], [141], while FedMalDE [142] further slices the extracted FCG into class-level *Sliced Graphs* for downstream graph learning. In contrast, EveDroid [143] applies FlowDroid to obtain event-level call graphs. Radare2 has also been employed to directly construct control flow graphs (CFGs) from binaries [144], [145]. Meanwhile, other works focus on source code analysis, where tools such as AstBuilder and ApkTool are used to build initial generic correlation graphs and APK-API graphs, respectively [146], [147].

[138] uses ApkTool extracts `smali` and `Manifest` files, and APIs with their code block IDs are parsed from the `smali` files. Key APIs are selected via linear regression to construct local API graphs, which are then merged into a global API graph and encoded by a GNN using five types of centrality features including degree, betweenness, closeness, eigenvector and PageRank. Permissions and intents are extracted from the `Manifest` as additional features. In the analysis stage, the graph embeddings are combined with permission and intent features and fed into a classifier for benign/malicious classification.

Compared with API graphs, function call graphs (FCGs) provide broader coverage and offer a more comprehensive representation of a program’s overall behavior. Moreover, the printable string information (PSI)-Graph derived from FCGs can accurately capture device-to-device communication patterns. PSI-Graph [139] employs deep learning techniques to detect IoT botnets by extracting high-level features from FCGs. Specifically, a customized `IDAPython` plug-in is used to extract both the function call graph and the printable string information (PSI) from an unpacked or native ELF binary. The PSIs contain critical information reflecting the life cycle of IoT botnets, such as IP addresses of other vulnerable devices, username–password patterns, malicious code injection commands, and C&C server addresses with their malicious instructions. Based on this, the method constructs a PSI-Graph, where nodes represent functions containing PSIs and edges denote caller–callee relationships. To capture the structural characteristics, the PSI-Graph is embedded into a

1024-dimensional vector using `graph2vec` [148], which is subsequently fed into a trained convolutional neural network (CNN) to classify the sample as either malicious or benign.

In existing research, after obtaining the FCG, TransMalDE [140] and MDHE [141] focus on sensitive API-centric subgraphs to accelerate program behavior analysis. They first label sensitive API calls in the FCG and then compute the maliciousness degree of each API using the Term Frequency–Inverse Document Frequency (TF–IDF) method, where the term frequency corresponds to the proportion of malware samples invoking the API, and the inverse document frequency is the logarithmic inverse of the total number of malware and benign samples invoking it. Subsequently, a k -hop neighborhood subgraph is constructed for each sensitive API, and the maliciousness degree of the subgraph is defined as the sum of the maliciousness scores of all sensitive APIs it contains. To reflect the varying importance of different subgraphs, both TransMalDE and MDHE assign weights to each malicious subgraph using sum normalization ($f_i / \sum f_i$), thereby obtaining the feature value for each subgraph and forming the subgraph feature vector set. The two approaches differ in their final classification stage: TransMalDE directly feeds the subgraph feature vector set into a trained Transformer-based malware detection model, whose Softmax layer outputs the detection result (malware or benign). In contrast, MDHE performs feature fusion by combining the subgraph maliciousness features with permission features, and the resulting final feature vector is fed into a pre-trained Capsule Network (CapsNet)–based classifier to determine whether the APK is benign or malicious.

Similarly, FedMalDE [142] slices the extracted FCG into class-level *Sliced Graphs* (SGs) through a sub-function call graph-based representation, dividing the entire FCG into n subgraphs. It then selects the top- k SGs with the highest semantic relevance as *Sliced Subgraphs* (SSG) for malware detection. In each Sliced Graph, a node (representing a class) aggregates features from all methods within that class, including **Parameter Features** (e.g., the number of method parameters), **Sensitive API Features**, **Instruction Features** (e.g., the types of instructions used), and **Modifier Features** (e.g., access modifiers and non-access modifiers). Each edge attribute represents the shortest calling path between two

different classes. For feature representation, FedMalDE applies a word embedding method to project the textual information of the SSG into an embedding vector space. The resulting vectors are then fed into a *Subgraph Aggregated Capsule Network* (SACN), which uses a Sigmoid classifier to output the final prediction, determining whether the sample is malicious or benign.

In [143], EveDroid first employs FlowDroid to extract event-level call graphs from an Android application. The root of each graph is a special method named `dummyMainMethod`, whose child nodes correspond to life-cycle callbacks, GUI events, or system events. Starting from each event entry point, EveDroid traverses the corresponding subgraph to obtain the API call sequence for that event. Each API call is then decomposed into a sequence of words, including the package name, return type, method name, and parameter list. Using the `doc2vec` tool from NLP, these textual components are mapped into a semantic vector space. Subsequently, EveDroid applies the K-means clustering algorithm to assign each API call vector to a specific semantic category. For each event, the frequency distribution of its API call vectors over the categories is computed to form the event's feature representation. Finally, the concatenation of all event feature vectors constitutes the behavioral feature vector of the entire application. This event group vector is fed into a neural network (NN), whose Softmax layer outputs the final classification result.

In contrast, [144] process iot program via Radare2 Python API (r2pipe) to extract CFGs $G = (V, E)$. Then it derives two feature categories: (i) general features—number of nodes/edges, graph density, and component count; (ii) graph algorithm features—degree centrality, shortest path, closeness/betweenness centrality, diameter, and radius. Results show IoT malware generally has fewer nodes, more edges, higher density, and a single component compared to Android malware. Finally, 23 CFG features (in seven groups) are fed into LR, SVM, RF, and CNN models, evaluating the performance of these models. Similarly, DL-FHMC [145] uses Radare2 to extract a program's control flow graph and compute 23 graph features (e.g., centrality, shortest path, density, node/edge counts), forming a $1 \times 231 \times 23$ vector for the detection model, which outputs benign or malicious via softmax. Malicious samples are further classified into Gafgyt, Mirai, or Tsunami families, while benign samples proceed to the suspicious behavior detection module. This module mines common subgraph patterns for each family using gSpan, ranks them by size, frequency, coverage, and inverse frequency, and selects the top 10,000 per family (30,000 total). A 30,000-dimensional one-hot feature vector is then generated using the VF2 subgraph isomorphism algorithm and fed into a machine learning model to determine the presence of suspicious behavior; positive cases are forwarded to dynamic analysis or security experts for further investigation.

Unlike the aforementioned studies that parse a apk or binary file into a graph, Niffler[146] parses event-triggering logic from open-source SmartApp code, uses `AstBuilder`

to build an initial generic correlation graph, infers actuator–sensor dependencies via a Naïve Bayes model, and eliminates orthogonal, irrelevant sensor associations using a Markov chain. Next, by incorporating real-time traffic data, it applies k -means clustering (with the Elbow method to select k) to refine device grouping, dynamically generating high-precision device-specific correlation graphs. In the detection phase, the dynamic graph stream is represented as a time-series edge set; a biased random walk combined with Node2vec is employed to extract high-sensitivity link paths, where sensitivity is quantified using the Levenshtein distance, retaining only the most likely anomalous interaction paths. Finally, for each device, Niffler builds a local inference model based on the top- k correlated neighbors selected by a GNN and mRMR, predicts the expected device state, and flags it as anomalous if the deviation from the actual state exceeds a predefined threshold.

GCDroid [147] first uses ApkTool to decompile given APKs, obtaining its corresponding resource files and smali files. By analyzing the smali source code, the system extracts the API call information of the APK. Based on this information, it constructs an APK–API heterogeneous graph, where the two types of nodes represent APKs and officially designated sensitive APIs, respectively, and the edges (APK–API) indicate invocation relationships between them. The authors then propose a Graph Compression with Reachable Relation (GCRR) method to transform the heterogeneous APK–API graph into a homogeneous APK–APK graph (adjacency matrix). In this compressed graph, an edge between two APK nodes signifies that they invoke the same API, with edge weights reflecting the strength of their association. Node features are derived from the original APK–API graph, including statistics such as the number of API calls, the proportion of sensitive API calls, and structural connectivity metrics. Finally, the resulting APK–APK graph, along with its node features, is fed into a Graph Convolutional Network (GCN) for node classification, where each APK node is labeled as either malware or benign.

2) *malware classification*: In malware classification, various types of graphs have been employed to represent either device communication behaviors or program execution behaviors. A communication graph is typically constructed from traffic data to model the traffic flow between different IP addresses. In contrast, function call graphs (FCGs) and control flow graphs (CFGs) capture program behaviors at different levels of granularity, with FCGs representing interactions between functions and CFGs depicting control transfers at the basic block level.

In [149], a graph feature–based machine learning approach is proposed for botnet traffic classification. The approach first ingests network traffic generated by botnets and constructs a communication graph using Graph-tool. In this graph, each node represents a unique IP address, while each directed edge denotes a communication flow between two IP addresses. The edge weights are determined by the number of transmitted packets. Graph-tool is also employed to com-

pute three categories of graph features: Degree-based features, Centrality-based features (e.g., betweenness, closeness, PageRank), and the Clustering feature (i.e., Local Clustering Coefficient, LCC). After that, the approach applies filter-based feature selection techniques, including four evaluation measures: Information Gain (IG) / Mutual Information, Gini Impurity (GI), Correlation-based Measure (CM), and Consistency-based Measure (CBF). This process yields a subset of features that are highly correlated with botnet behaviors. Finally, the selected features are fed into multiple supervised classifiers—Naïve Bayes (NB), Decision Tree (DT), Random Forests (RF), AdaBoost, Extra Trees Classifier (ETC), and K-Nearest Neighbors (KNN)—to detect and classify different botnet families.

Unlike communication graphs that are constructed from traffic data to capture interactions among IoT devices, MaGraMal [150] is based on masked graph representation and is designed to identify fine-grained malicious behaviors of IoT malware during the lurking stage. For each malware sample, MaGraMal leverages Radare2 to extract its function call graph (FCG). To reduce redundancy and noise, the method selects the node with the highest degree centrality as the central node, computes the shortest paths from this node to all others, and adaptively determines the optimal mask radius. Using the central node and mask radius, a masked graph is constructed by filtering out peripheral nodes. The masked graph is then embedded into a vector representation using `graph2vec`, while three additional structural features—number of nodes, number of edges, and graph density—are also extracted. These features are combined into a $(d+3)$ -dimensional vector, where d denotes the dimension of the `graph2vec` embedding. Finally, the resulting vector is input into a trained classification model that assigns four fine-grained malicious behavior labels for the lurking stage: persistence, privilege escalation, deception, and defense evasion.

In contrast to the aforementioned studies, both DL-FHMC [151] and Soteria [152] first employ Radare2 to extract control flow graphs (CFGs) from binaries, and then either derive high-level features or extract representative subgraphs. These features or subgraphs are subsequently transformed into vector embeddings to serve as inputs to learning models. More specifically, DL-FHMC extracts 23 features across six categories—betweenness centrality, closeness centrality, degree centrality, shortest path, edge properties, and node properties—which are represented as 23-dimensional vectors. These vectors are then fed into trained classifiers, including RF, DNN, and CNN, for malware detection. In contrast, Soteria introduces two labeling strategies, Density-Based Labeling (DBL) and Level-Based Labeling (LBL), to generate annotated CFGs. Random walks are then performed on each graph to generate path sequences, from which subsequences of varying lengths are extracted using the n -gram method. Sequence frequencies are computed, and the top 500 most frequent sequences from both DBL- and LBL-labeled graphs are selected as features, yielding a 1000-dimensional feature vector. This feature vector is input into a five-layer autoencoder

to determine whether the sample is malicious. Once a sample is classified as malware, DL-FHMC directly feeds the feature vector into the trained classifiers, whereas Soteria extracts an additional 20 feature vectors (10 from DBL and 10 from LBL). These vectors are input into two separate Convolutional Neural Networks (CNNs) to generate class probability distributions, and the final malware classification is determined using a majority voting scheme. The key distinction lies in how the two approaches handle benign samples. If a sample is classified as benign, DL-FHMC further analyzes its feature vector using the Suspicious Behavior Detector, which applies the `gSpan` algorithm with weighted ranking to mine frequent subgraphs from malware families and leverages the VF2 subgraph isomorphism algorithm to identify whether these malicious patterns exist in the CFG, thereby flagging potential adversarial examples. Soteria, on the other hand, performs no further analysis once a sample is deemed benign.

3) *Lessons learned:* Our survey demonstrates that graph learning representations can be effectively applied to IoT malware detection and classification. In the following, we highlight several key lessons derived from this study.

- Graph types differ in granularity, which directly impacts construction cost, analysis complexity, and the accuracy of detection or classification. API graphs are the most lightweight and easy to build, capturing only API invocation relations and thus supporting fast but less accurate detection. Function Call Graphs (FCGs) capture caller–callee relations among functions, enabling broader behavioral analysis with higher accuracy but at greater construction difficulty. Control Flow Graphs (CFGs), based on basic blocks, provide the most fine-grained execution semantics, offering the highest accuracy while being the most complex to construct and analyze. Beyond program-level graphs, device-specific correlation graphs derived from source code and traffic-based graphs from communication flows provide coarse-grained representations of device interactions; although they cannot capture internal program semantics, they effectively model device-to-device behavior with lower complexity. The overall comparison of different graph features is shown in Table VI.
- Current mainstream methods for feature extraction and embedding can be roughly divided into three categories. The first directly embeds graphs into a vector space using methods such as `graph2vec`, but the resulting vectors often contain substantial noise, which increases the difficulty of classification or detection for learning-based models. To address this, some studies attempt to extract more representative subgraphs to reduce redundant noise; however, these algorithms are computationally expensive, and balancing subgraph size with analysis efficiency remains an open challenge. The second approach extracts features such as centrality, paths, and node/edge counts from graphs and then maps them into a vector space. This method effectively captures the global properties of graphs while offering lower computational

complexity and better interpretability. The third approach combines structural information with semantic contextual information by fusing their vector representations, producing richer features that significantly enhance detection performance.

- Existing graph learning-based methods are primarily designed for IoT Android or ELF binaries, and their migration to other architectures remains challenging.
- Practical IoT applications require real-time detection; however, current graph analysis-based approaches are computationally intensive and incur high detection overhead, making them difficult to deploy effectively in real-time IoT environments.

TABLE VI
GRAPH FEATURE EXTRACTION METHODS FOR MALWARE ANALYSIS

Paper	Feature Engineering	Performance
API Graph		
[138]	ApkTool	98.5%
Call Graph		
[139]	IDA Pro	97.0%
[150]	Radare2	81.7%
[140]	Androguard + Gephi	98.2%
[141]	Androguard + Gephi	96.2 %
[142]	-	97.2%,96.7%
[143]	FlowDroid	99.0%
Control Flow Graph		
[144]	Radare2	99.5%
[145]	Radare2	97.4%, 97.7%, 97.6%
[151]	Radare2	97.4%, 97.7%, 97.6%
[152]	Radare2	-
Traffic-based Graph		
[149]	Graph-tool	-
Correlation Graph		
[146]	AstBuilder, correlation analysis	-
APK-API Graph		
[147]	ApkTool,graph analysis	-

IV. FUTURE DIRECTIONS

A. Static Analysis Feature

Overall, most existing approaches rely on traditional static analysis tools to disassemble programs and extract assembly-level information such as opcodes and control-flow graphs (CFGs). These features are then further processed through temporal modeling, similarity computation, statistical analysis, image generation, or feature fusion. In contrast, methods that leverage deep learning to directly learn feature representations remain limited, making the integration of deep learning-based feature extraction a promising future direction.

In terms of program visualization, current static methods primarily generate images using strategies such as byte-to-pixel mapping, proc-dump-based imaging, or semantic extraction with IDA Pro to construct RGB or Markov images. Similarly, disassembly tools are employed to obtain CFGs, which are then used to model program behaviors. However, research on graph-based features remains largely confined to CFGs with similarity analysis. In the future, combining non-learning analytical methods with diverse graph representation

techniques may offer more efficient solutions for IoT malware detection and classification.

Finally, we observe a clear trend from single-feature analysis toward multi-feature fusion. Exploring the integration of different static features and their complementary strengths therefore represents another promising direction for advancing IoT malware analysis.

B. Dynamic Analysis Feature

Current research primarily focuses on **traffic-based features**, where various methods extract features from network traffic for malicious behavior detection and classification. However, this line of work often remains limited to a single traffic modality; future studies could combine traffic with other dynamic features to improve coverage and robustness. Meanwhile, some efforts already derive **runtime context** or **hardware features** as numeric representations, or transform them into **visual modalities** (e.g., images, spectrograms, graphs) to characterize IoT system behavior. Yet most studies still emphasize selecting specific numeric features or visualizing them as images and spectrograms; leveraging **graph-structured** representations of system behavior remains comparatively underexplored and merits further investigation.

C. Hybrid Analysis Feature

Existing research has primarily focused on leveraging static and dynamic contextual information for IoT malware detection, while the use of other feature types and their associated techniques remains largely unexplored. Moreover, most current approaches emphasize hybrid contextual features for malware detection, whereas the application of hybrid features to IoT malware classification still represents an open research direction.

D. Graph Learning Representation Feature

Graph-based learning methods in IoT malware analysis primarily focus on call graphs and control flow graphs (CFGs), while only a few studies leverage alternative representations such as API graphs or traffic-based graphs. Most existing approaches rely on tools such as ApkTool, Radare2, or IDA Pro to extract graphs directly from raw bytecode or binaries. Therefore, improving the accuracy and fidelity of these graph extraction techniques represents a potential research direction for enhancing detection and classification performance. At the same time, to reduce the complexity of graph analysis, some studies have begun to extract subgraphs for downstream malware analysis. However, balancing the trade-off between subgraph size and the richness of program semantics it preserves remains an open challenge. Furthermore, integrating graph representations with other contextual program features to construct richer hybrid features for behavior modeling is another promising but underexplored direction.

V. CONCLUSION

Feature selection and extraction techniques form the cornerstone of IoT malware analysis, as effective feature engineering

can significantly enhance the accuracy and robustness of malware detection and classification. A key challenge in this field lies in identifying the most effective features and corresponding analysis methods across diverse IoT environments. Addressing this challenge requires continuous investigation into the performance of different feature extraction techniques, as well as the design and optimization of combinations of features with various malware analysis approaches. In recent years, researchers have explored a wide range of such combinations to improve detection and classification outcomes. In this survey, we systematically categorize, compare, and analyze these approaches, and summarize the lessons learned. Furthermore, we release the dataset we collected and propose potential future feature directions to guide subsequent research in this domain.

REFERENCES

- [1] Global Growth Insights. Global iot (internet of things) market size report. <https://www.globalgrowthinsights.com/market-reports/iot-market-112820>, 2025. Accessed: Aug. 30, 2025.
- [2] Cogniteq. The history of iot: How the technology is evolving. <https://www.cogniteq.com/blog/history-iot-how-technology-evolving#2087>, 2024. Accessed: Aug. 30, 2025.
- [3] Muhammad Adil, Houbing Song, Spyridon Mastorakis, Hussein Abulkasim, Ahmed Farouk, and Zhanpeng Jin. Uav-assisted iot applications, cybersecurity threats, ai-enabled solutions, open challenges with future research directions. *IEEE Transactions on Intelligent Vehicles*, 9(4):4583–4605, 2023.
- [4] Fatima Hussain, Rasheed Hussain, Syed Ali Hassan, and Ekram Hossain. Machine learning in iot security: Current solutions and future challenges. *IEEE Communications Surveys & Tutorials*, 22(3):1686–1721, 2020.
- [5] Madumitha Venkatasubramanian, Arash Habibi Lashkari, and Saqib Hakak. Iot malware analysis using federated learning: A comprehensive survey. *IEEE Access*, 11:5004–5018, 2023.
- [6] Ori Or-Meir, Nir Nissim, Yuval Elovici, and Lior Rokach. Dynamic malware analysis in the modern era—a state of the art survey. *ACM Computing Surveys (CSUR)*, 52(5):1–48, 2019.
- [7] Yanfang Ye, Tao Li, Donald Adjero, and S Sitharama Iyengar. A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)*, 50(3):1–40, 2017.
- [8] Matthew G Gaber, Mohiuddin Ahmed, and Helge Janicke. Malware detection with artificial intelligence: A systematic literature review. *ACM Computing Surveys*, 56(6):1–33, 2024.
- [9] Anandharaju Durai Raju, Ibrahim Y Abualhaol, Ronnie Salvador Giagone, Yang Zhou, and Shengqiang Huang. A survey on cross-architectural iot malware threat hunting. *IEEE Access*, 9:91686–91709, 2021.
- [10] Akshara Ravi, Vivek Chaturvedi, and Muhammad Shafique. Vit4mal: Lightweight vision transformer for malware detection on edge devices. *ACM Trans. Embed. Comput. Syst.*, 22(5s), September 2023.
- [11] Lakshmanan Nataraj, Sreejith Karthikeyan, Gregoire Jacob, and Bangalore S Manjunath. Malware images: visualization and automatic classification. In *Proceedings of the 8th international symposium on visualization for cyber security*, pages 1–7, 2011.
- [12] Mohamed Abdel-Basset, Hossam Hawash, Karam M. Sallam, Ibrahim Elgendi, Kumudu Munasinghe, and Abbas Jamalipour. Efficient and lightweight convolutional networks for iot malware detection: A federated learning approach. *IEEE Internet of Things Journal*, 10(8):7164–7173, 2023.
- [13] Amir Namavar Jahromi, Sattar Hashemi, Ali Dehghantanha, Reza M. Parizi, and Kim-Kwang Raymond Choo. An enhanced stacked lstm method with no random initialization for malware threat hunting in safety and time-critical systems. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 4(5):630–640, 2020.
- [14] Danish Vasan, Mamoun Alazab, Sitalakshmi Venkatraman, Junaid Akram, and Zheng Qin. Mthael: Cross-architecture iot malware detection based on neural network advanced ensemble learning. *IEEE Transactions on Computers*, 69(11):1654–1667, 2020.
- [15] Gaofeng Zhang, Yu Li, Xudan Bao, Chinmay Chakarborty, Joel JPC Rodrigues, Liping Zheng, Xuyun Zhang, Lianyong Qi, and Mohammad R Khosravi. Tsdroid: A novel android malware detection framework based on temporal & spatial metrics in iomt. *ACM Transactions on Sensor Networks*, 19(3):1–23, 2023.
- [16] Hisham Alasmay, Afsah Anwar, Ahmed Abusnaina, Abdulrahman Alabduljabbar, Mohammed Abuhamad, An Wang, Daehun Nyang, Amro Awad, and David Mohaisen. Shellcore: Automating malicious iot software detection using shell commands representation. *IEEE Internet of Things Journal*, 9(4):2485–2496, 2021.
- [17] Rahim Taheri, Mohammad Shojafar, Mamoun Alazab, and Rahim Tafazolli. Fed-iiot: A robust federated malware detection architecture in industrial iot. *IEEE Transactions on Industrial Informatics*, 17(12):8442–8452, 2021.
- [18] Xiaoheng Deng, Xinjun Pei, Shengwei Tian, and Lan Zhang. Edge-based iiot malware detection for mobile devices with offloading. *IEEE Transactions on Industrial Informatics*, 19(7):8093–8103, 2023.
- [19] ElMouatez Billah Karbab, Mourad Debbabi, Abdelouahid Derhab, and Djedjiga Mouheb. Cypider: building community-based cyber-defense infrastructure for android malware detection. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*, pages 348–362, 2016.
- [20] Zhuocheng Yu, Shudong Li, Youming Bai, Weihong Han, Xiaobo Wu, and Zhihong Tian. Remsf: a robust ensemble model of malware detection based on semantic feature fusion. *IEEE Internet of Things Journal*, 10(18):16134–16143, 2023.
- [21] Peng Wu, Dong Liu, Junfeng Wang, Baoguo Yuan, and Wenyuan Kuang. Detection of fake iot app based on multidimensional similarity. *IEEE Internet of Things Journal*, 7(8):7021–7031, 2020.
- [22] Remus M Petrache, Ciprian Oprisa, and Camelia Lemnar. Unveiling hidden structures: Multi-platform iot malware analysis using graph embeddings. *IEEE Access*, 2025.
- [23] Yunchun Zhang, Zikun Liao, Ning Zhang, Shaohui Min, Qi Wang, Tony QS Quek, and Mingxiong Zhao. Deep hashing for malware family classification and new malware identification. *IEEE Internet of Things Journal*, 2024.
- [24] Farhan Ullah, Gautam Srivastava, Shamsher Ullah, and Leonardo Mostarda. Privacy-preserving federated learning approach for distributed malware attacks with intermittent clients and image representation. *IEEE Transactions on Consumer Electronics*, 2023.
- [25] Qi Li, Jiaxin Mi, Weishi Li, Junfeng Wang, and Mingyu Cheng. Cnn-based malware variants detection method for internet of things. *IEEE Internet of Things Journal*, 8(23):16946–16962, 2021.
- [26] Jueun Jeon, Byeonghui Jeong, Seungyeon Baek, and Young-Sik Jeong. Static multi feature-based malware detection using multi spp-net in smart iot environments. *IEEE Transactions on Information Forensics and Security*, 2024.
- [27] Baoguo Yuan, Junfeng Wang, Peng Wu, and Xianguo Qing. Iot malware classification based on lightweight convolutional neural networks. *IEEE Internet of Things Journal*, 9(5):3770–3783, 2022.
- [28] Dhanya K. A., Vinod P., Suleiman Y. Yerima, Abul Bashar, Anwin David, Abhiram T., Alan Antony, Ashil K. Shavanas, and Gireesh Kumar T. Obfuscated malware detection in iot android applications using markov images and cnn. *IEEE Systems Journal*, 17(2):2756–2766, 2023.
- [29] Changren Mai, Riqing Liao, Jing Ren, Yuanxiang Gong, Kaibo Zhang, and Chiya Zhang. Mobilenet-based iot malware detection with opcode features. *Journal of Communications and Information Networks*, 8(3):221–230, 2023.
- [30] Yuhao Xie, Xiong Luo, and Jiankun Sun. Towards enhancing sequence-optimized malware representation with context-separated bi-directional long short-term memory and proximal policy optimization. *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [31] Mirabelle Dib, Sadeh Torabi, Elias Bou-Harb, Nizar Bouguila, and Chadi Assi. Evoliot: A self-supervised contrastive learning framework for detecting and characterizing evolving iot malware variants. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pages 452–466, 2022.
- [32] Yanchen Qiao, Weizhe Zhang, Xiaojiang Du, and Mohsen Guizani. Malware classification based on multilayer perception and word2vec for iot security. *ACM Transactions on Internet Technology (TOIT)*, 22(1):1–22, 2021.
- [33] Shudong Li, Yuan Li, Xiaobo Wu, Sattam Al Otaibi, and Zhihong Tian. Imbalanced malware family classification using multimodal fusion and

- weight self-learning. *IEEE Transactions on Intelligent Transportation Systems*, 24(7):7642–7652, 2022.
- [34] Mohammadhadi Alaeiyan, Ali Dehghantanha, Tooska Dargahi, Mauro Conti, and Saeed Parsa. A multilabel fuzzy relevance clustering system for malware attack attribution in the edge layer of cyber-physical networks. *ACM Transactions on Cyber-Physical Systems*, 4(3):1–22, 2020.
- [35] Chin-Wei Tien, Shang-Wen Chen, Tao Ban, and Sy-Yen Kuo. Machine learning framework to analyze iot malware using elf and opcode features. *Digital Threats: Research and Practice*, 1(1):1–19, 2020.
- [36] Mirabelle Dib, Sadegh Torabi, Elias Bou-Harb, and Chadi Assi. A multi-dimensional deep learning framework for iot malware classification and family attribution. *IEEE Transactions on Network and Service Management*, 18(2):1165–1177, 2021.
- [37] Marcin Woźniak, Jakub Silka, Michał Wiecek, and Mubarak Al-rashoud. Recurrent neural network model for iot and networking malware threat detection. *IEEE Transactions on Industrial Informatics*, 17(8):5583–5594, 2021.
- [38] Nadra Guizani and Arif Ghafoor. A network function virtualization system for detecting malware in large iot based networks. *IEEE Journal on Selected Areas in Communications*, 38(6):1218–1228, 2020.
- [39] Dingde Jiang, Zhihao Wang, Ye Wang, Lizhuang Tan, Jian Wang, and Peiying Zhang. A blockchain-reinforced federated intrusion detection architecture for iiot. *IEEE Internet of Things Journal*, 11(16):26793–26805, 2024.
- [40] Guru Prasad Bhandari, Andreas Lyth, Andrii Shalaginov, and Tor-Morten Grønli. Artificial intelligence enabled middleware for distributed cyberattacks detection in iot-based smart environments. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 3023–3032. IEEE, 2022.
- [41] Yangyang Mei, Weihong Han, Shudong Li, Kaihan Lin, Zhihong Tian, and Shumei Li. A novel network forensic framework for advanced persistent threat attack attribution through deep learning. *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [42] Daniel Uhříček, Karel Hynek, Tomáš Čejka, and Dušan Kolář. Bota: Explainable iot malware detection in large networks. *IEEE Internet of Things Journal*, 10(10):8416–8431, 2023.
- [43] Ibbad Hafeez, Markku Antikainen, Aaron Yi Ding, and Sasu Tarkoma. Iot-keeper: Detecting malicious iot network activity using online traffic analysis at the edge. *IEEE Transactions on Network and Service Management*, 17(1):45–59, 2020.
- [44] Arvin Hekmati, Jiahe Zhang, Tamoghna Sarkar, Nishant Jethwa, Eugenio Grippo, and Bhaskar Krishnamachari. Correlation-aware neural networks for ddos attack detection in iot systems. *IEEE/ACM Transactions on Networking*, 2024.
- [45] Yu-Wei Chang, Hong-Yen Chen, Chansu Han, Tomohiro Morikawa, Takeshi Takahashi, and Tsung-Nan Lin. Finish: Efficient and scalable nmf-based federated learning for detecting malware activities. *IEEE Transactions on Emerging Topics in Computing*, 2023.
- [46] Yair Meidan, Dan Avraham, Hanan Libhaber, and Asaf Shabtai. Cadesh: Collaborative anomaly detection for smart homes. *IEEE Internet of Things Journal*, 10(10):8514–8532, 2022.
- [47] Sajid Ali, Omar Abusabha, Farman Ali, Muhammad Imran, and Tamer Abuhmed. Effective multitask deep learning for iot malware detection and identification using behavioral traffic analysis. *IEEE Transactions on Network and Service Management*, 20(2):1199–1209, 2023.
- [48] Nour Moustafa, Benjamin Turnbull, and Kim-Kwang Raymond Choo. An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things. *IEEE Internet of Things Journal*, 6(3):4815–4830, 2018.
- [49] Weina Niu, Jian'An Xiao, Xiyue Zhang, Xiaosong Zhang, Xiaojiang Du, Xiaoming Huang, and Mohsen Guizani. Malware on internet of uavs detection combining string matching and fourier transformation. *IEEE Internet of Things Journal*, 8(12):9905–9919, 2021.
- [50] Nagarathna Ravi and S Mercy Shalinie. Learning-driven detection and mitigation of ddos attack in iot via sdn-cloud architecture. *IEEE Internet of Things Journal*, 7(4):3559–3570, 2020.
- [51] Shigen Shen, Longjun Huang, Haiping Zhou, Shui Yu, En Fan, and Qiying Cao. Multistage signaling game-based optimal detection strategies for suppressing malware diffusion in fog-cloud-based iot networks. *IEEE Internet of Things Journal*, 5(2):1043–1054, 2018.
- [52] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, and Ahmad-Reza Sadeghi. DIot: A federated self-learning anomaly detection system for iot. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pages 756–767, 2019.
- [53] Kalupahana Liyanage Kushan Sudheera, Dinil Mon Divakaran, Rhishi Pratap Singh, and Mohan Gurusamy. Adept: Detection and identification of correlated attack stages in iot networks. *IEEE Internet of Things Journal*, 8(8):6591–6607, 2021.
- [54] Tianyang Cai, Tao Jia, Sridhar Adepu, Yuqi Li, and Zheng Yang. Adam: an adaptive ddos attack mitigation scheme in software-defined cyber-physical system. *IEEE Transactions on Industrial Informatics*, 19(6):7802–7813, 2023.
- [55] Mingshu He, Yuanming Huang, Xinlei Wang, Peng Wei, and Xiaojuan Wang. A lightweight and efficient iot intrusion detection method based on feature grouping. *IEEE Internet of Things Journal*, 11(2):2935–2949, 2023.
- [56] Richard Mitev, Anna Pazii, Markus Miettinen, William Enck, and Ahmad-Reza Sadeghi. Leakypick: Iot audio spy detector. In *Proceedings of the 36th Annual Computer Security Applications Conference*, pages 694–705, 2020.
- [57] Ragav Sridharan, Rajib Ranjan Maiti, and Nils Ole Tippenhauer. Wadac: Privacy-preserving anomaly detection and attack classification on wireless traffic. In *Proceedings of the 11th ACM conference on security & privacy in wireless and mobile networks*, pages 51–62, 2018.
- [58] Chunjing Han, Tong Li, Qiuyi Chen, Yulei Wu, and Jifeng Qin. Distributed and collaborative lightweight edge federated learning for iot zombie devices detection. *ACM Transactions on Sensor Networks*, 2024.
- [59] Love Allen Chijioke Ahakonye, Cosmas Ifeanyi Nwakanma, Jae-Min Lee, and Dong-Seong Kim. Trees bootstrap aggregation for detection and characterization of iot-scada network traffic. *IEEE Transactions on Industrial Informatics*, 20(4):5217–5228, 2024.
- [60] Mohammad Al-Fawa'reh, Jumana Abu-Khalaf, Patryk Szewczyk, and James Jin Kang. Malbot-drl: Malware botnet detection using deep reinforcement learning in iot networks. *IEEE Internet of Things Journal*, 11(6):9610–9629, 2024.
- [61] Osama Alkadi, Nour Moustafa, Benjamin Turnbull, and Kim-Kwang Raymond Choo. A deep blockchain framework-enabled collaborative intrusion detection for protecting iot and cloud networks. *IEEE Internet of Things Journal*, 8(12):9463–9472, 2020.
- [62] Manisha Malik, Maitreyee Dutta, Jorge Granjal, et al. Iot-sentry: A cross-layer-based intrusion detection system in standardized internet of things. *IEEE Sensors Journal*, 21(24):28066–28076, 2021.
- [63] Ahamed Aljuhani, Prabhat Kumar, Rehab Alanazi, Turki Albalawi, Okba Taouali, AKM Najmul Islam, Neeraj Kumar, and Mamoun Alazab. A deep-learning-integrated blockchain framework for securing industrial iot. *IEEE Internet of Things Journal*, 11(5):7817–7827, 2023.
- [64] Love Allen Chijioke Ahakonye, Gabriel Chukwunonso Amaizu, Cosmas Ifeanyi Nwakanma, Jae Min Lee, and Dong-Seong Kim. Classification and characterization of encoded traffic in scada network using hybrid deep learning scheme. *Journal of Communications and Networks*, 26(1):65–79, 2024.
- [65] Amandeep Kaur. Intrusion detection approach for industrial internet of things traffic using deep recurrent reinforcement learning assisted federated learning. *IEEE Transactions on Artificial Intelligence*, 2024.
- [66] Ruoyu Li, Qing Li, Jianer Zhou, and Yong Jiang. Adriot: An edge-assisted anomaly detection framework against iot-based network attacks. *IEEE Internet of Things Journal*, 9(13):10576–10587, 2021.
- [67] Mojtaba Eskandari, Zaffar Haider Janjua, Massimo Vecchio, and Fabio Antonelli. Passban ids: An intelligent anomaly-based intrusion detection system for iot edge devices. *IEEE Internet of Things Journal*, 7(8):6882–6897, 2020.
- [68] Dina Ayesha S., Siddique A.B., and Manivannan D. Fs3: Few-shot and self-supervised framework for efficient intrusion detection in internet of things networks. In *Proceedings of the 39th Annual Computer Security Applications Conference, ACSAC '23*, page 138–149, New York, NY, USA, 2023. Association for Computing Machinery.
- [69] Ravi Vinayakumar, Mamoun Alazab, Sriram Srinivasan, Quoc-Viet Pham, Soman Kotti Padannayil, and Ketha Simran. A visualized botnet detection system based deep learning for the internet of things networks of smart cities. *IEEE Transactions on Industry Applications*, 56(4):4436–4456, 2020.
- [70] Lihua Yin, Xi Luo, Chunsheng Zhu, Liming Wang, Zhen Xu, and Hui Lu. Connspoiler: Disrupting c&c communication of iot-based botnet through fast detection of anomalous domain queries. *IEEE Transactions on Industrial Informatics*, 16(2):1373–1384, 2019.

- [71] Abraham Peedikayil Kuruvila, Shamik Kundu, and Kanad Basu. Defending hardware-based malware detectors against adversarial attacks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(9):1727–1739, 2021.
- [72] John Demme, Matthew Maycock, Jared Schmitz, Adrian Tang, Adam Waksman, Simha Sethumadhavan, and Salvatore Stolfo. On the feasibility of online malware detection with performance counters. *ACM SIGARCH computer architecture news*, 41(3):559–570, 2013.
- [73] Sai Praveen Kadiyala, Manaar Alam, Yash Shrivastava, Sikhar Patranabis, Muhamed Fauzi Bin Abbas, Arnab Kumar Biswas, Debdeep Mukhopadhyay, and Thambipillai Srikanthan. Lambda: Lightweight assessment of malware for embedded architectures. *ACM Transactions on Embedded Computing Systems (TECS)*, 19(4):1–31, 2020.
- [74] Rana Elnaggar, Kanad Basu, Krishnendu Chakrabarty, and Ramesh Karri. Runtime malware detection using embedded trace buffers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(1):35–48, 2022.
- [75] Zhixin Pan, Jennifer Sheldon, and Prabhat Mishra. Hardware-assisted malware detection and localization using explainable machine learning. *IEEE Transactions on Computers*, 71(12):3308–3321, 2022.
- [76] Kamran Rahmani, Prabhat Mishra, and Sandip Ray. Efficient trace signal selection using augmentation and ilp techniques. In *Fifteenth International Symposium on Quality Electronic Design*, pages 148–155, 2014.
- [77] Robin Joe Prabhakar Soundar Raja James, Abdurhman Ali Albasir, Kshirasagar Naik, Marzia Zaman, and Nishith Goel. A power signal based dynamic approach to detecting anomalous behavior in wireless devices. In *Proceedings of the 16th ACM International Symposium on Mobility Management and Wireless Access*, pages 9–18, 2018.
- [78] Zhuoran Li and Danella Zhao. Zerod-fender: A resource-aware iot malware detection engine via fine-grained side-channel analysis. *ACM Transactions on Design Automation of Electronic Systems*, 29(6):1–25, 2024.
- [79] Nader Sehatbakhsh, Alireza Nazari, Haider Khan, Alenka Zajic, and Milos Prvulovic. Emma: Hardware/software attestation framework for embedded systems using electromagnetic signals. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 983–995, 2019.
- [80] Adrien Cosson, Amit Kumar Sikder, Leonardo Babun, Z Berkay Celik, Patrick McDaniel, and A Selcuk Uluagac. Sentinel: A robust intrusion detection system for iot networks using kernel-level system information. In *Proceedings of the International Conference on Internet-of-Things Design and Implementation*, pages 53–66, 2021.
- [81] Gyeongmin Lee, Bongjun Kim, Seungbin Song, Changsu Kim, Jong Kim, and Hanjun Kim. Precise correlation extraction for iot fault detection with concurrent activities. *ACM Trans. Embed. Comput. Syst.*, 2021.
- [82] Keshav Sood, Mohammad Reza Nosouhi, Neeraj Kumar, Anuroop Gaddam, Bohao Feng, and Shui Yu. Accurate detection of iot sensor behaviors in legitimate, faulty and compromised scenarios. *IEEE Transactions on Dependable and Secure Computing*, 20(1):288–300, 2021.
- [83] Amir Namavar Jahromi, Hadis Karimipour, Ali Dehghantanha, and Kim-Kwang Raymond Choo. Toward detection and attribution of cyber-attacks in iot-enabled cyber-physical systems. *IEEE Internet of Things Journal*, 8(17):13712–13722, 2021.
- [84] Saurav Chakraborty, Agnieszka Onuchowska, Sagar Samtani, Wolfgang Jank, and Brandon Wolfram. Machine learning for automated industrial iot attack detection: An efficiency-complexity trade-off. *ACM Transactions on Management Information System (TMIS)*, 12(4):1–28, 2021.
- [85] Mariam MN Aboelwafa, Karim G Seddik, Mohamed H Eldefrawy, Yasser Gadallah, and Mikael Gidlund. A machine-learning-based technique for false data injection attacks detection in industrial iot. *IEEE Internet of Things Journal*, 7(9):8462–8471, 2020.
- [86] Anand Mudgerikar, Puneet Sharma, and Elisa Bertino. Edge-based intrusion detection for iot devices. *ACM Transactions on Management Information Systems (TMIS)*, 11(4):1–21, 2020.
- [87] Jiwei Zhang, Md Zakirul Alam Bhuiyan, Xu Yang, Tian Wang, Xuesong Xu, Thaier Hayajneh, and Faiza Khan. Anticoncealer: Reliable detection of adversary concealed behaviors in edgeai-assisted iot. *IEEE Internet of Things Journal*, 9(22):22184–22193, 2021.
- [88] Donpiti Chulerttiyawong and Abbas Jamalipour. Sybil attack detection in internet of flying things-ioft: A machine learning approach. *IEEE Internet of Things Journal*, 10(14):12854–12866, 2023.
- [89] João B Borges, Joao PS Medeiros, Luiz PA Barbosa, Heitor S Ramos, and Antonio AF Loureiro. Iot botnet detection based on anomalies of multiscale time series dynamics. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12282–12294, 2022.
- [90] Ali Davanian, Michail Faloutsos, and Martina Lindorfer. C2miner: Tricking iot malware into revealing live command & control servers. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ASIA CCS ’24, page 112–127, New York, NY, USA, 2024. Association for Computing Machinery.
- [91] Amit Kumar Sikder, Leonardo Babun, and A Selcuk Uluagac. Aegis+ a context-aware platform-independent security framework for smart home systems. *Digital Threats: Research and Practice*, 2(1):1–33, 2021.
- [92] Hania Mohamed, Nickolaos Koroniotis, and Nour Moustafa. Digital forensics based on federated learning in iot environment. In *Proceedings of the 2023 Australasian Computer Science Week*, ACSW ’23, page 92–101, New York, NY, USA, 2023. Association for Computing Machinery.
- [93] Ke Xu, Xixi Zhang, Yu Wang, Tomoaki Ohtsuki, Bamidele Adebisi, Hikmet Sari, and Guan Gui. Self-supervised learning malware traffic classification based on masked autoencoder. *IEEE Internet of Things Journal*, 11(10):17330–17340, 2024.
- [94] Woosub Jung, Kailai Cui, Kenneth Koltermann, Junjie Wang, Chun-Sheng Xin, and Gang Zhou. Light auditor: power measurement can tell private data leakage through iot covert channels. In *Proceedings of the 20th ACM conference on embedded networked sensor systems*, pages 518–532, 2022.
- [95] Abdurhman Albasir, Kshirasagar Naik, and Ricardo Manzano. Toward improving the security of iot and cps devices: An ai approach. *Digital Threats: Research and Practice*, 4(2):1–30, 2023.
- [96] Sreenitha Kasarapu, Sanket Shukla, and Sai Manoj Pudukotai Dinakar-rao. Resource-and workload-aware model parallelism-inspired novel malware detection for iot devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(12):4618–4628, 2023.
- [97] Bardia Esmaeili, Amin Azmoodeh, Ali Dehghantanha, Hadis Karimipour, Behrouz Zolfaghari, and Mohammad Hammoudeh. Iiot deep malware threat hunting: from adversarial example detection to adversarial scenario detection. *IEEE Transactions on Industrial Informatics*, 18(12):8477–8486, 2022.
- [98] Nader Sehatbakhsh, Alireza Nazari, Monjur Alam, Frank Werner, Yuanda Zhu, Alenka Zajic, and Milos Prvulovic. Remote: Robust external malware detection framework by using electromagnetic signals. *IEEE Transactions on Computers*, 69(3):312–326, 2020.
- [99] Nikhil Chawla, Harshit Kumar, and Saibal Mukhopadhyay. Machine learning in wavelet domain for electromagnetic emission based malware analysis. *IEEE Transactions on Information Forensics and Security*, 16:3426–3441, 2021.
- [100] Alireza Nazari, Nader Sehatbakhsh, Monjur Alam, Alenka Zajic, and Milos Prvulovic. Eddie: Em-based detection of deviations in program execution. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pages 333–346, 2017.
- [101] Antonio Libri, Andrea Bartolini, and Luca Benini. paella: Edge ai-based real-time malware detection in data centers. *IEEE Internet of Things Journal*, 7(10):9589–9599, 2020.
- [102] Fei Ding, Hongda Li, Feng Luo, Hongxin Hu, Long Cheng, Hai Xiao, and Rong Ge. Deeppower: Non-intrusive and deep learning-based detection of iot malware using power side channels. In *Proceedings of the 15th ACM Asia conference on computer and communications security*, pages 33–46, 2020.
- [103] Ramyapandian Vijayakanthan, Karley M Waguespack, Irfan Ahmed, and Aisha Ali-Gombe. Fortifying iot devices: Ai-driven intrusion detection via memory-encoded audio signals. In *2023 IEEE Secure Development Conference (SecDev)*, pages 106–117. IEEE, 2023.
- [104] Xueying Han, Susu Cui, Jian Qin, Song Liu, Bo Jiang, Cong Dong, Zhigang Lu, and Baoxu Liu. Contramtd: An unsupervised malicious network traffic detection method based on contrastive learning. In *Proceedings of the ACM Web Conference 2024*, pages 1680–1689, 2024.
- [105] Ramesh Paudel, Timothy Muncy, and William Eberle. Detecting dos attack in smart home iot devices using a graph-based approach. In *2019 IEEE international conference on big data (big data)*, pages 5249–5258. IEEE, 2019.

- [106] Antonino Rullo, Daniele Midi, Anand Mudjerikar, and Elisa Bertino. Kalis2.0—a secaaas-based context-aware self-adaptive intrusion detection system for iot. *IEEE Internet of Things Journal*, 11(7):12579–12601, 2023.
- [107] Tobias Wüchner, Martín Ochoa, and Alexander Pretschner. Malware detection with quantitative data flow graphs. In *Proceedings of the 9th ACM symposium on Information, computer and communications security*, pages 271–282, 2014.
- [108] Fei Tong, Cheng Chen, and Jianping Pan. A novel detection and localization scheme for wormhole attack in internet of things. *IEEE Internet of Things Journal*, 11(4):7141–7152, 2023.
- [109] Xiaotian Ma, Yipeng Wang, Yingxu Lai, Wenxu Jia, Zijian Zhao, Huijie He, Ruiping Yin, and Yige Chen. A multi-perspective feature approach to few-shot classification of iot traffic. *IEEE Transactions on Green Communications and Networking*, 7(4):2052–2066, 2023.
- [110] Tooba Hasan, Jahanzaib Malik, Iram Bibi, Wali Ullah Khan, Fahd N Al-Wesabi, Kapal Dev, and Gaojian Huang. Securing industrial internet of things against botnet attacks using hybrid deep learning approach. *IEEE Transactions on Network Science and Engineering*, 10(5):2952–2963, 2022.
- [111] Hichem Sedjelmaci, Nesrine Kaaniche, Aymen Boudguiga, and Nirwan Ansari. Secure attack detection framework for hierarchical 6g-enabled internet of vehicles. *IEEE Transactions on Vehicular Technology*, 73(2):2633–2642, 2023.
- [112] Muhammad Shafiq, Zhihong Tian, Ali Kashif Bashir, Xiaojiang Du, and Mohsen Guizani. Corrauc: a malicious bot-iot traffic detection method in iot network using machine-learning techniques. *IEEE Internet of Things Journal*, 8(5):3242–3254, 2020.
- [113] Ilias Siniosoglou, Panagiotis Radoglou-Grammatikis, Georgios Efstathiopoulos, Panagiotis Fouliras, and Panagiotis Sarigiannidis. A unified deep learning anomaly detection and classification approach for smart grid environments. *IEEE Transactions on Network and Service Management*, 18(2):1137–1151, 2021.
- [114] Alec F Diallo and Paul Patras. Adaptive clustering-based malicious traffic classification at the network edge. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2021.
- [115] Partha Pratim Kundu, Tram Truong-Huu, Ling Chen, Luying Zhou, and Sin G Teo. Detection and classification of botnet traffic using deep learning with model explanation. *IEEE Transactions on Dependable and Secure Computing*, 2022.
- [116] Min Lu, Bin Zhou, and Zhiyong Bu. Two-stage distillation-aware compressed models for traffic classification. *IEEE Internet of Things Journal*, 10(16):14152–14166, 2023.
- [117] Yawar Abbas Abid, Jinsong Wu, Guangquan Xu, Shihui Fu, and Muhammad Waqas. Multi-level deep neural network for distributed denial-of-service attack detection and classification in software-defined networking supported internet of things networks. *IEEE Internet of Things Journal*, 2024.
- [118] Shizhou Zhu, Xiaolong Xu, Juan Zhao, and Fu Xiao. Lkd-stnn: A lightweight malicious traffic detection method for internet of things based on knowledge distillation. *IEEE Internet of Things Journal*, 11(4):6438–6453, 2023.
- [119] Jianbang Dai, Xiaolong Xu, Honghao Gao, and Fu Xiao. Cmftc: Cross modality fusion efficient multitask encrypt traffic classification in iiot environment. *IEEE Transactions on Network Science and Engineering*, 10(6):3989–4009, 2023.
- [120] Ons Aouedi, Kandaraj Piamrat, Guillaume Muller, and Kamal Singh. Federated semisupervised learning for attack detection in industrial internet of things. *IEEE Transactions on Industrial Informatics*, 19(1):286–295, 2022.
- [121] Qingjun Yuan, Gaopeng Gou, Yanbei Zhu, Yuefei Zhu, Gang Xiong, and Yongjuan Wang. Mcre: A unified framework for handling malicious traffic with noise labels based on multidimensional constraint representation. *IEEE Transactions on Information Forensics and Security*, 19:133–147, 2023.
- [122] Xixi Zhang, Qin Wang, Maoyang Qin, Yu Wang, Tomoaki Ohtsuki, Bamidele Adebisi, Hikmet Sari, and Guan Gui. Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search. *IEEE Transactions on Information Forensics and Security*, 2024.
- [123] Xixi Zhang, Liang Hao, Guan Gui, Yu Wang, Bamidele Adebisi, and Hikmet Sari. An automatic and efficient malware traffic classification method for secure internet of things. *IEEE Internet of Things Journal*, 11(5):8448–8458, 2024.
- [124] Jinhui Ning, Guan Gui, Yu Wang, Jie Yang, Bamidele Adebisi, Song Ci, Haris Gacanin, and Fumiyuki Adachi. Malware traffic classification using domain adaptation and ladder network for secure industrial internet of things. *IEEE Internet of Things Journal*, 9(18):17058–17069, 2022.
- [125] Yuehua Huo, Wei Liang, Junhan Chen, Shangyuan Zhuang, and Jiyan Sun. Lightguard: A lightweight malicious traffic detection method for internet of things. *IEEE Internet of Things Journal*, 2024.
- [126] Xiaoyan Hu, Cheng Zhu, Guang Cheng, Ruidong Li, Hua Wu, and Jian Gong. A deep subdomain adaptation network with attention mechanism for malware variant traffic identification at an iot edge gateway. *IEEE Internet of Things Journal*, 10(5):3814–3826, 2023.
- [127] Duy-Phuc Pham, Damien Marion, Matthieu Mastio, and Annelie Heuser. Obfuscation revealed: Leveraging electromagnetic signals for obfuscated malware classification. In *Proceedings of the 37th Annual Computer Security Applications Conference*, pages 706–719, 2021.
- [128] Yunkai Bai, Andrew Stern, Jungmin Park, Mark Tehranipoor, and Domenic Forte. Rascv2: Enabling remote access to side-channels for mission critical and iot systems. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 27(6):1–25, 2022.
- [129] Amit Kumar Sikder, Leonardo Babun, Hidayet Aksu, and A Selcuk Uluagac. Aegis: A context-aware security framework for smart home systems. In *Proceedings of the 35th Annual Computer Security Applications Conference*, pages 28–41, 2019.
- [130] Sixing Lu and Roman Lysecky. Time and sequence integrated runtime anomaly detection for embedded systems. *ACM Transactions on Embedded Computing Systems (TECS)*, 17(2):1–27, 2017.
- [131] Wei Zhang, Yan Meng, Yugeng Liu, Xiaokuan Zhang, Yinqian Zhang, and Haojin Zhu. Homonit: Monitoring smart home apps from encrypted traffic. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1074–1088, 2018.
- [132] Mohammad Wazid, Ashok Kumar Das, and Sachin Shetty. Bsfr-sh: Blockchain-enabled security framework against ransomware attacks for smart healthcare. *IEEE Transactions on Consumer Electronics*, 69(1):18–28, 2022.
- [133] Mahbub E Khoda, Tasadduq Imam, Joarder Kamruzzaman, Iqbal Gondal, and Ashfaqur Rahman. Robust malware defense in industrial iot applications using machine learning with selective adversarial samples. *IEEE Transactions on Industry Applications*, 56(4):4415–4424, 2020.
- [134] Muhammad Junaid Iqbal, Sana Aurangzeb, Muhammad Aleem, Gautam Srivastava, and Jerry Chun-Wei Lin. Rthreatdroid: A ransomware detection approach to secure iot based healthcare systems. *IEEE Transactions on Network Science and Engineering*, 10(5):2574–2583, 2022.
- [135] Aolin Ding, Praveen Murthy, Luis Garcia, Pengfei Sun, Matthew Chan, and Saman Zonouz. Mini-me, you complete me! data-driven drone security via dnn-based approximate computing. In *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, pages 428–441, 2021.
- [136] Xiaoheng Deng, Bin Chen, Xuechen Chen, Xinjun Pei, Shaohua Wan, and Sotirios K Goudos. A trusted edge computing system based on intelligent risk detection for smart iot. *IEEE Transactions on Industrial Informatics*, 20(2):1445–1454, 2023.
- [137] Usman Ahmed, Jerry Chun-Wei Lin, Gautam Srivastava, and Alireza Jolfaei. Active learning based adversary evasion attacks defense for malwares in the internet of things. *IEEE Systems Journal*, 17(2):2434–2444, 2023.
- [138] Rahul Yumlembam, Biju Issac, Seibu Mary Jacob, and Longzhi Yang. Iot-based android malware detection using graph neural network with adversarial defense. *IEEE Internet of Things Journal*, 10(10):8432–8444, 2023.
- [139] Huy-Trung Nguyen, Quoc-Dung Ngo, and Van-Hoang Le. A novel graph-based approach for iot botnet detection. *International Journal of Information Security*, 19(5):567–577, 2020.
- [140] Xiaoheng Deng, Zhe Wang, Xinjun Pei, and Kaiping Xue. Transmalde: An effective transformer based hierarchical framework for iot malware detection. *IEEE Transactions on Network Science and Engineering*, 11(1):140–151, 2024.
- [141] Xiaoheng Deng, Haowen Tang, Xinjun Pei, Deng Li, and Kaiping Xue. Mdhe: A malware detection system based on trust hybrid user-edge

evaluation in iot network. *IEEE Transactions on Information Forensics and Security*, 18:5950–5963, 2023.

- [142] Xinjun Pei, Xiaoheng Deng, Shengwei Tian, Lan Zhang, and Kaiping Xue. A knowledge transfer-based semi-supervised federated learning for iot malware detection. *IEEE Transactions on Dependable and Secure Computing*, 20(3):2127–2143, 2023.
- [143] Tao Lei, Zhan Qin, Zhibo Wang, Qi Li, and Dengpan Ye. Evedroid: Event-aware android malware detection against model degrading for iot devices. *IEEE Internet of Things Journal*, 6(4):6668–6680, 2019.
- [144] Hisham Alasmmary, Aminollah Khormali, Afsah Anwar, Jeman Park, Jinchun Choi, Ahmed Abusnaina, Amro Awad, Daehun Nyang, and Aziz Mohaisen. Analyzing and detecting emerging internet of things malware: A graph-based approach. *IEEE Internet of Things Journal*, 6(5):8977–8988, 2019.
- [145] Ahmed Abusnaina, Mohammed Abuhamad, Hisham Alasmmary, Afsah Anwar, Rhongho Jang, Saeed Salem, DaeHun Nyang, and David Mohaisen. Dl-fhmc: Deep learning-based fine-grained hierarchical learning approach for robust malware classification. *IEEE Transactions on Dependable and Secure Computing*, 19(5):3432–3447, 2022.
- [146] Haohua Du, Yue Wang, Xiaoya Xu, and Mingsheng Liu. Niffler: Real-time device-level anomalies detection in smart home. *ACM Transactions on the Web*, 17(3):1–27, 2023.
- [147] Weina Niu, Yihang Wang, Xingyu Liu, Ran Yan, Xiong Li, and Xiaosong Zhang. Gcdroid: Android malware detection based on graph compression with reachability relationship extraction for iot devices. *IEEE Internet of Things Journal*, 10(13):11343–11356, 2023.
- [148] Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. graph2vec: Learning distributed representations of graphs. *arXiv preprint arXiv:1707.05005*, 2017.
- [149] Afnan Alharbi and Khalid Alsubhi. Botnet detection approach using graph-based machine learning. *Ieee Access*, 9:99166–99180, 2021.
- [150] Ruitao Feng, Sen Li, Sen Chen, Mengmeng Ge, Xuwei Li, and Xiaohong Li. Unmasking the lurking: Malicious behavior detection for iot malware with multi-label classification. In *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*, pages 95–106, 2024.
- [151] Ahmed Abusnaina, Mohammed Abuhamad, Hisham Alasmmary, Afsah Anwar, Rhongho Jang, Saeed Salem, DaeHun Nyang, and David Mohaisen. Dl-fhmc: Deep learning-based fine-grained hierarchical learning approach for robust malware classification. *IEEE Transactions on Dependable and Secure Computing*, 19(5):3432–3447, 2021.
- [152] Hisham Alasmmary, Ahmed Abusnaina, Rhongho Jang, Mohammed Abuhamad, Afsah Anwar, DaeHun Nyang, and David Mohaisen. Soteria: Detecting adversarial examples in control flow graph-based malware classifiers. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, pages 888–898, 2020.